Ola : Ensemble Learning

Ola, a leading ride-sharing platform in India, is committed to delivering reliable, affordable, and convenient urban transportation. The success of its operations heavily relies on a stable and satisfied base of drivers.

Introduction: The goal is to analyze historical driver data to identify behavioral and operational patterns that indicate potential churn. By understanding factors like performance, income, city, quarterly ratings, and education, Ola can forecast which drivers are at risk of leaving the platform.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
import warnings
warnings.filterwarnings('ignore')
```

```
! wget https://d2beiqkhq929f0.cloudfront.net/public_assets/assets/000/002/492/original/ola_driver_scaler.csv
```

```
--2025-08-01 18:50:00--  https://d2beiqkhq929f0.cloudfront.net/public_assets/assets/000/002/492/original/ola_driver_scaler.csv
Resolving d2beiqkhq929f0.cloudfront.net (d2beiqkhq929f0.cloudfront.net)... 13.224.9.181, 13.224.9.129, 13.224.9.24, ...
Connecting to d2beiqkhq929f0.cloudfront.net (d2beiqkhq929f0.cloudfront.net)|13.224.9.181|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1127673 (1.1M) [text/plain]
Saving to: ‘ola_driver_scaler.csv.1’

ola_driver_scaler.c 100%[===================>]   1.08M  --.-KB/s    in 0.05s

2025-08-01 18:50:00 (19.8 MB/s) - ‘ola_driver_scaler.csv.1’ saved [1127673/1127673]
```

```
df= pd.read_csv('ola_driver_scaler.csv', index_col=0 )
df.head()
```

	MMM-YY	Driver_ID	Age	Gender	City	Education_Level	Income	Dateofjoining	LastWorkingDate	Joining Designation	Grade	Total Business Value	Quarter Rati
0	01/01/19	1	28.0	0.0	C23	2	57387	24/12/18	NaN	1	1	2381060	
1	02/01/19	1	28.0	0.0	C23	2	57387	24/12/18	NaN	1	1	-665480	
2	03/01/19	1	28.0	0.0	C23	2	57387	24/12/18	03/11/19	1	1	0	
3	11/01/20	2	31.0	0.0	C7	2	67016	11/06/20	NaN	2	2	0	

Next steps: Generate code with df | View recommended plots | New interactive sheet

EDA:

```
df.shape
```

```
(19104, 13)
```

- The dataset has 19,104 rows and 13 columns.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 19104 entries, 0 to 19103
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   MMM-YY                19104 non-null  object
 1   Driver_ID             19104 non-null  int64
 2   Age                   19043 non-null  float64
```

```
 3   Gender                19052 non-null  float64
 4   City                  19104 non-null  object
 5   Education_Level       19104 non-null  int64
 6   Income                19104 non-null  int64
 7   Dateofjoining         19104 non-null  object
 8   LastWorkingDate       1616 non-null   object
 9   Joining Designation   19104 non-null  int64
 10  Grade                 19104 non-null  int64
 11  Total Business Value  19104 non-null  int64
 12  Quarterly Rating      19104 non-null  int64
dtypes: float64(2), int64(7), object(4)
memory usage: 2.5+ MB
```

The database has 13 columns, in which:

- Driver_ID, Education_Level, Income, Joining Designation, Grade, Total Business Value, and Quarterly Rating are of int64 data type.
- Age and Gender are of float64 data type.
- MMM-YY, City, Dateofjoining, and LastWorkingDate are of object data type.

```
# Missing values
df.isna().sum()
```

	0
MMM-YY	0
Driver_ID	0
Age	61
Gender	52
City	0
Education_Level	0
Income	0
Dateofjoining	0
LastWorkingDate	17488
Joining Designation	0
Grade	0
Total Business Value	0
Quarterly Rating	0

dtype: int64

The column Age has 61 null values, Gender has 52 null values, and LastWorkingDate has 17,488 null values.

```
# Dublicate values
df.duplicated().sum()
```

```
np.int64(0)
```

The dataset has no duplicate rows

```
# Describtion of all numerical feature
df.describe()
```

	Driver_ID	Age	Gender	Education_Level	Income	Joining Designation	Grade	Total Business Value	Quarterly Rating
count	19104.000000	19043.000000	19052.000000	19104.000000	19104.000000	19104.000000	19104.000000	1.910400e+04	19104.000000
mean	1415.591133	34.668435	0.418749	1.021671	65652.025126	1.690536	2.252670	5.716621e+05	2.008899
std	810.705321	6.257912	0.493367	0.800167	30914.515344	0.836984	1.026512	1.128312e+06	1.009832
min	1.000000	21.000000	0.000000	0.000000	10747.000000	1.000000	1.000000	-6.000000e+06	1.000000
25%	710.000000	30.000000	0.000000	0.000000	42383.000000	1.000000	1.000000	0.000000e+00	1.000000
50%	1417.000000	34.000000	0.000000	1.000000	60087.000000	1.000000	2.000000	2.500000e+05	2.000000
75%	2137.000000	39.000000	1.000000	2.000000	83969.000000	2.000000	3.000000	6.997000e+05	3.000000

- The average Age of drivers is approximately 34.67 years, with a minimum of 21 and a maximum of 58.
- The Gender column: 0 represents male and 1 represents female. With a mean of 0.418, the data indicates a higher proportion of male drivers.
- The average Income is around ₹65,652, ranging from ₹10,747 to ₹188,418, with a standard deviation of ₹30,914, suggesting moderate variation in earnings.
- The Education_Level is categorized as 0 for 10+, 1 for 12+, and 2 for Graduate. The average education level is approximately 1.02, indicating that most drivers have education beyond 10th grade.
- The Joining Designation ranges from 1 to 5, with a mean of 1.69, indicating that most drivers start from a similar entry-level position.
- The Grade column (1 to 5) likely reflects driver performance, with an average grade of 2.25.
- The Total Business Value column ranges from −6,000,000 to 33,747,720, with a mean of ₹571,662. The negative values may indicate losses or refunds, and the large range suggests the presence of outliers.
- The Quarterly Rating ranges from 1 to 5, where 5 is the best rating. The average rating is 2.01, suggesting that most drivers are rated between 1 and 3.

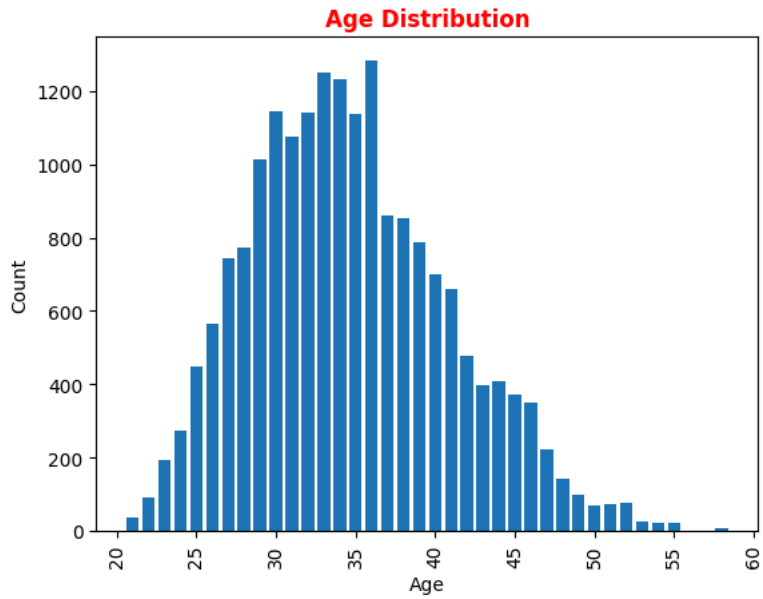
Data type Convertion: Several columns in the dataset require data type conversion for proper analysis:

- MMM-YY, Dateofjoining, and LastWorkingDate are currently of object type but should be converted to datetime format.
- Gender is stored as float64, but since it is a binary categorical variable (0 for male, 1 for female), it should be converted to category type.
- Education_Level, Joining Designation, Grade, and Quarterly Rating are encoded as int64 but represent categorical values; therefore, they should be converted to the category data type.

```
# Datetime Converstion
df['MMM-YY'] = pd.to_datetime(df['MMM-YY'])
df['Dateofjoining']= pd.to_datetime(df['Dateofjoining'])
df['LastWorkingDate']= pd.to_datetime(df['LastWorkingDate'])

# Converting float and integer columns into categorical data types.
df['Gender'] = df['Gender'].astype('category')
df['Education_Level'] = df['Education_Level'].astype('category')
df['Joining Designation'] = df['Joining Designation'].astype('category')
#df['Grade'] = df['Grade'].astype('category')
```

Univariant Analysis:

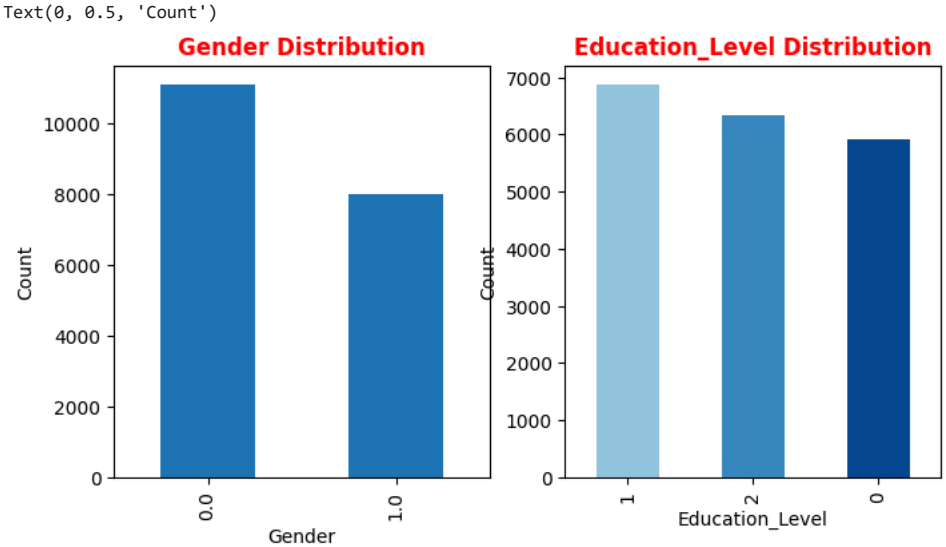
```
# The Distribution of Age
plt.bar(df['Age'].value_counts().index, df['Age'].value_counts().values)
plt.xticks(rotation=90)
plt.xlabel('Age')
plt.ylabel('Count')
plt.title('Age Distribution', fontweight='bold', color='red')
plt.show()
```



Age is approximately normally distributed with a slight right skew. Most drivers fall within the age range of 29 to 36, with a few outliers above the age of 50.

```
plt.figure(figsize=(8, 4))
plt.subplot(1, 2, 1)
# The Distribution of derivers Gender
df["Gender"].value_counts().plot(kind="bar")
plt.title("Gender Distribution", fontweight="bold", color='red')
plt.xlabel("Gender")
plt.ylabel("Count")

plt.subplot(1, 2, 2)
# The Distribution of Education_Level
cmap = plt.cm.Blues
colors = cmap(np.linspace(0.4, 0.9, df["Education_Level"].value_counts().shape[0]))
df["Education_Level"].value_counts().plot(kind="bar", color=colors)
plt.title("Education_Level Distribution", fontweight="bold", color='red')
plt.xlabel("Education_Level")
plt.ylabel("Count")
```

```
Text(0, 0.5, 'Count')
```

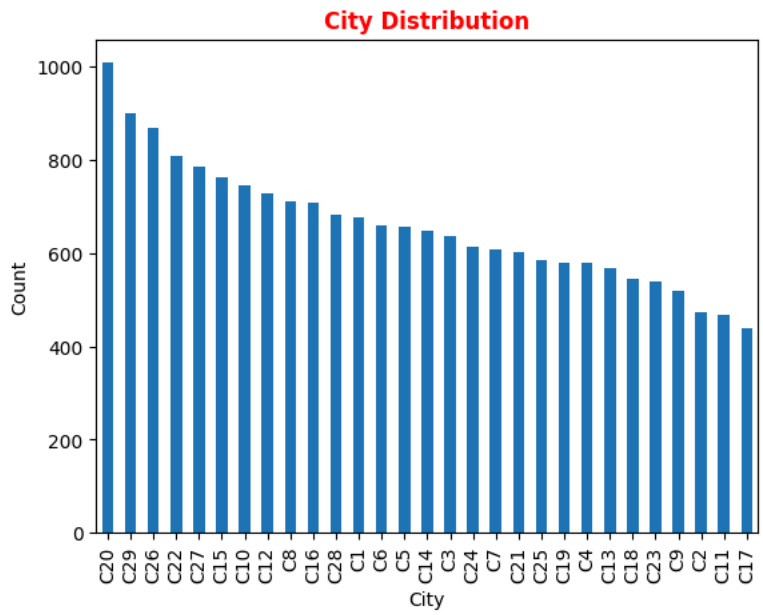


The gender distribution in the dataset shows that approximately 58.13% of the drivers are male (Gender = 0) and 41.87% are female (Gender = 1). This indicates a slightly higher representation of male drivers in the dataset.

The education level distribution shows that 35.93% of drivers have 12+ education (Education_Level = 1), followed by 33.12% who are graduates (Education_Level = 2), and 30.95% with 10+ education (Education_Level = 0). This indicates that a majority of drivers have education beyond 10th grade, with a fairly balanced distribution across all three categories.

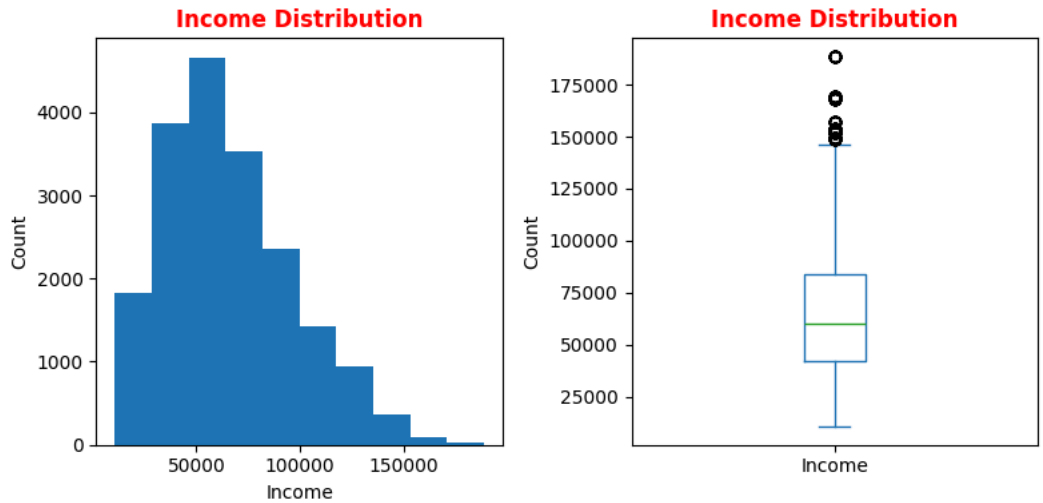
```
# The plot for Driver's count per City
df["City"].value_counts().plot(kind="bar")
plt.title("City Distribution", fontweight="bold", color='red')
plt.xlabel("City")
plt.ylabel("Count")
```

```
Text(0, 0.5, 'Count')
```



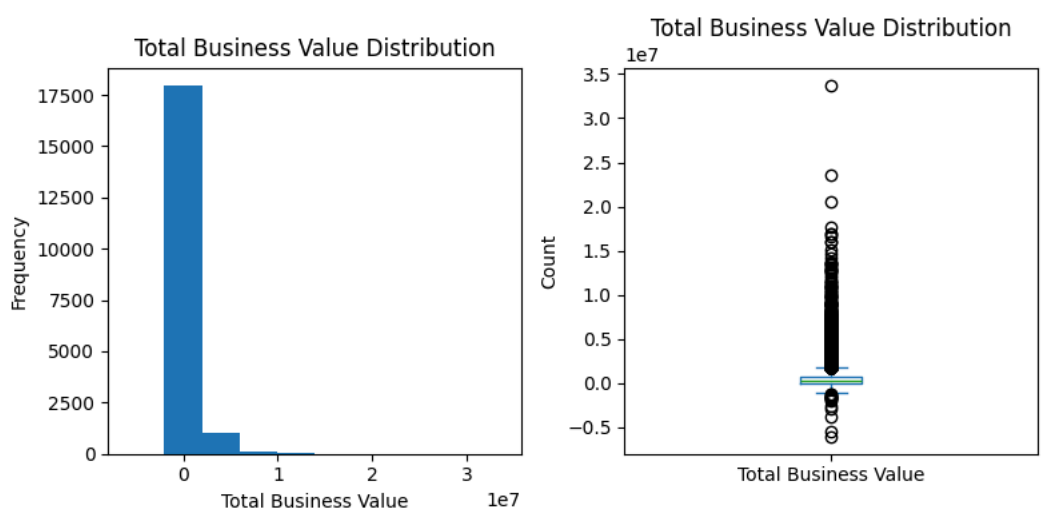
The highest representation from City C20 (5.28%), followed by C29 (4.71%), C26 (4.55%), C22 (4.23%). Cities like C9, C2, C11, and C17 have the lowest proportions, each contributing less than 2.5%. Overall, the city-wise distribution is relatively balanced.

```
# Plot for Income distribution
plt.figure(figsize=(8, 4))
plt.subplot(1, 2, 1)
df["Income"].plot(kind="hist")
plt.title("Income Distribution", fontweight="bold", color='red')
plt.xlabel("Income")
plt.ylabel("Count")

plt.subplot(1, 2, 2)
df["Income"].plot(kind="box")
plt.title("Income Distribution", fontweight="bold", color='red')
plt.ylabel("Count")

plt.super_title = "Income Distribution"

plt.tight_layout()
```



Income is right-skewed, with an average of around ₹65,652 and the presence of some large positive outliers.

```
# Plot for Total Business Value
plt.figure(figsize=(8, 4))
plt.subplot(1, 2, 1)
df["Total Business Value"].plot(kind="hist")
plt.title("Total Business Value Distribution")
plt.xlabel("Total Business Value")

plt.subplot(1, 2, 2)
df["Total Business Value"].plot(kind="box")
plt.title("Total Business Value Distribution")
plt.ylabel("Count")

plt.tight_layout()
```



The Total Business Value is heavily right-skewed with both negative and positive outliers.

It has a mean of ₹5.72 lakhs and a high standard deviation of ₹11.28 lakhs, indicating large variability.

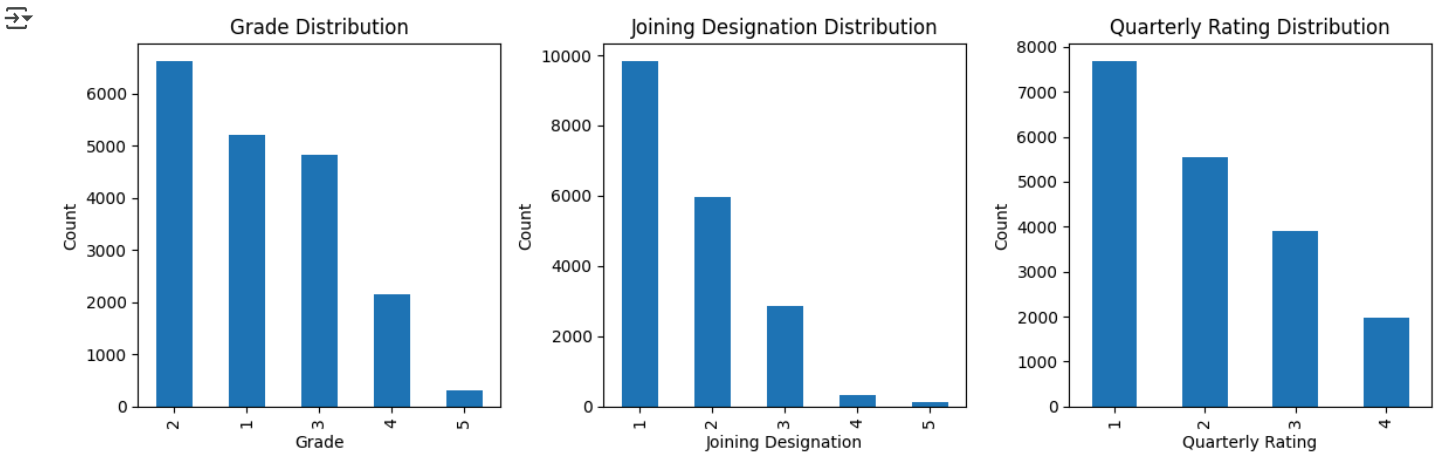
The values range from −₹60 lakhs to ₹3.37 crores, with a median of ₹2.5 lakhs.

75% of the drivers have a business value below ₹6.99 lakhs, while a few generate exceptionally high value.

The presence of negative values suggests some drivers incurred significant losses.

```
# Plot for Grade distibution of drivers Joining designation
plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
df["Grade"].value_counts().plot(kind="bar")
```

```
plt.title("Grade Distribution")
plt.xlabel("Grade")
plt.ylabel("Count")

# Plot for Joining designation distibution of drivers
plt.subplot(1, 3, 2)
df["Joining Designation"].value_counts().plot(kind="bar")
plt.title("Joining Designation Distribution")
plt.xlabel("Joining Designation")
plt.ylabel("Count")

# Plot for Quatarly Rating
plt.subplot(1, 3, 3)
df["Quarterly Rating"].value_counts().plot(kind="bar")
plt.title("Quarterly Rating Distribution")
plt.xlabel("Quarterly Rating")
plt.ylabel("Count")

plt.tight_layout()
plt.show()
```



Both Grade and Joining Designation distributions are right-skewed.

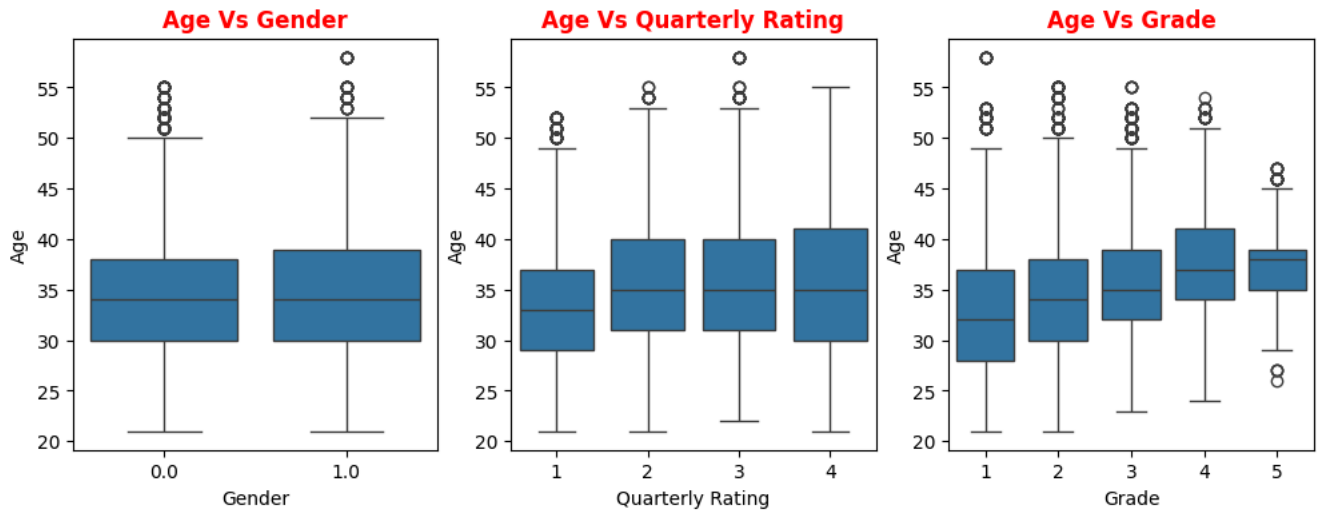
- The Grade distribution shows that the majority of drivers fall into Grade 2 (6627), followed by Grade 1 (5202) and Grade 3 (4826). Grade 4 has fewer drivers (2144), Grade 5 is very rare (just 305), indicating exceptional performance
- The Joining Designation distribution indicates that over half of the drivers (9831) started at Designation 1, followed by 35955 at Designation 2, and 2847 at Designation 3.
- The number of drivers decreases with increasing quarterly rating. Rating 1 is the most common among drivers, covering ~40% of the observations. Rating 4 is the least common, with only ~10% of drivers falling in this category.

Bivariant Analysis:

```
# Plot between Age Vs Gender
plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
sns.boxplot(data = df, x="Gender", y="Age" )
plt.title("Age Vs Gender", fontweight="bold", color="red")
plt.xlabel("Gender")
plt.ylabel("Age")

# Plot between Age Vs Quarterly Rating
plt.subplot(1, 3, 2)
sns.boxplot(data = df, x="Quarterly Rating", y="Age" )
plt.title("Age Vs Quarterly Rating", fontweight="bold", color="red")
plt.xlabel("Quarterly Rating")

# PLot between Age Vs Grade
```

```
plt.subplot(1, 3, 3)
sns.boxplot(data = df, x="Grade", y="Age" )
plt.title("Age Vs Grade", fontweight="bold", color="red")
plt.xlabel("Grade")
```

```
Text(0.5, 0, 'Grade')
```

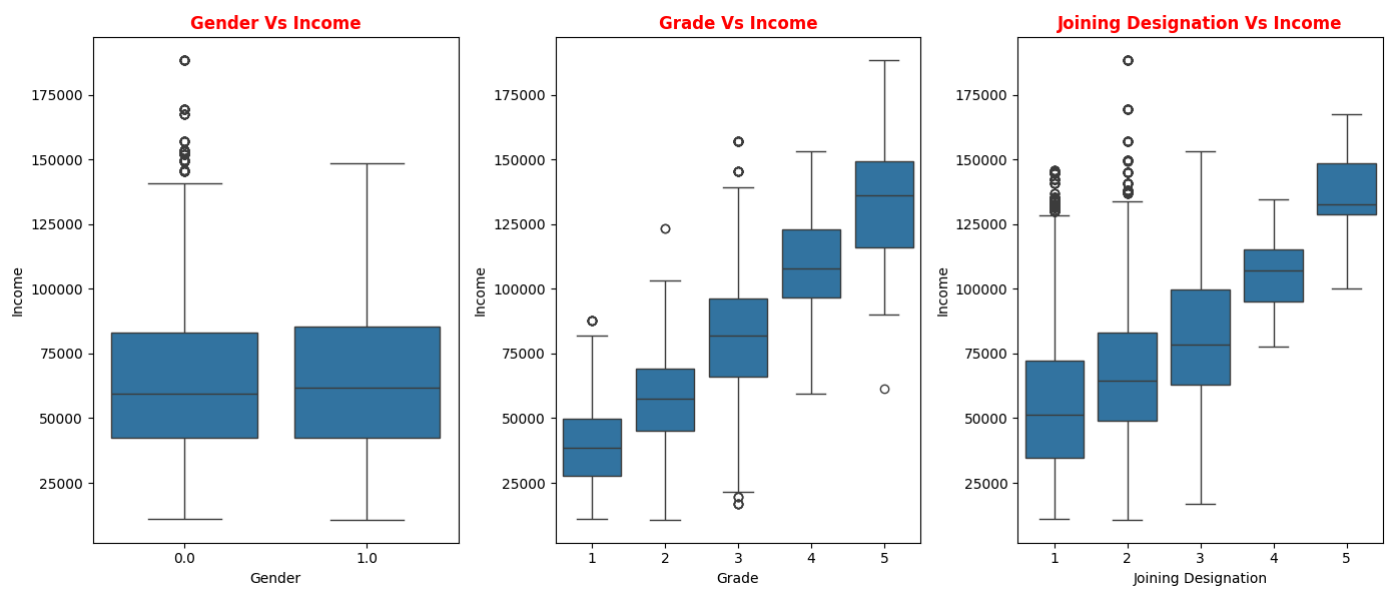


Age Vs Gender: The boxplot shows that the distribution of age for both genders is approximately the same, with a similar mean age of 34 and a minimum age of 21. However, the maximum age for females is slightly higher (58) compared to males (55). Both groups exhibit a few positive outliers in age.

Age Vs Quarterly Rating: The age distribution across different quarterly ratings shows a slight upward trend. Drivers with a rating of 1 have an average age of approximately 33 years. Drivers rated 2, 3, and 4 have mean ages of approximately 35, 36 years, respectively. The minimum age across all rating groups is around 21–22 years, while the maximum age ranges from 52 to 58 years

Age Vs Grade: The data shows a clear upward trend in average age with increasing driver Grade. Drivers in Grade 1 have the lowest average age of approximately 33 years, Drivers in Grade 5 have the highest average age of about 37 years. Similarly, The minimum age for Grade 1 is 21 years and for in Grade 5 is 26 years. The maximum age for grade 1 is 58 years and for grade 5 47 year.

```
# Plot for Income Vs Gende, Grade and Joining designation

# Gender vs Income
plt.figure(figsize=(14, 6))
plt.subplot(1, 3, 1)
sns.boxplot(data = df, x='Gender', y='Income')
plt.title('Gender Vs Income', fontweight='bold', color='red')
plt.xlabel('Gender')
plt.ylabel('Income')

# Grade vs Income
plt.subplot(1, 3, 2)
sns.boxplot(data = df, x='Grade', y='Income')
plt.title('Grade Vs Income', fontweight='bold', color='red')
plt.xlabel('Grade')

# Joining Designation Vs Income
plt.subplot(1, 3, 3)
sns.boxplot(data = df, x='Joining Designation', y='Income')
plt.title('Joining Designation Vs Income', fontweight='bold', color='red')
plt.xlabel('Joining Designation')

plt.tight_layout()
```



The income distribution is fairly similar across genders, with female drivers (Gender = 1.0) having a slightly higher average income of ₹66,159 compared to male drivers (Gender = 0.0), whose average income is ₹65,330.

The income distribution shows a clear upward trend across Grades 1 to 5, indicating that higher-grade drivers tend to earn more on average:

Grade 1 - average income of ₹39,006

Grade 2 - average of ₹57,211

Grade 3 - higher average income of ₹81,750

Grade 4 - average of ₹1,09,971.

Grade 5 - highest average income of ₹1,37,272.

The income distribution shows a clear upward trend with higher Joining Designations, indicating that drivers who join at higher designations tend to earn more.

Designation 1 - average income of ₹56,694.

Designation 2 - average income of ₹68,905.

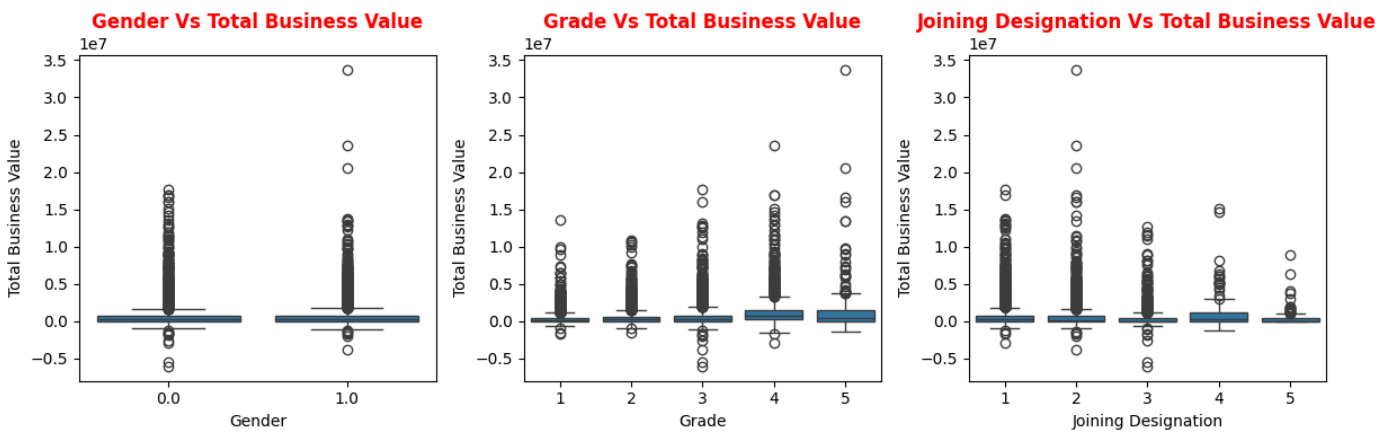
Designation 3 - has an average income of ₹81,805.

Designation 4 - average income of ₹1,06,439.

Designation 5 - highest average income of ₹1,33,332.

```
# Plot for  Total Business Value Vs Gende, Grade and Joining designation

# Gender Vs Total Business Value
plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
sns.boxplot(data = df, x='Gender', y='Total Business Value')
plt.title('Gender Vs Total Business Value', fontweight='bold', color='red')
plt.xlabel('Gender')
plt.ylabel('Total Business Value')

# Grade Vs Total Business Value
plt.subplot(1, 3, 2)
sns.boxplot(data = df, x='Grade', y='Total Business Value')
plt.title('Grade Vs Total Business Value', fontweight='bold', color='red')
plt.xlabel('Grade')
plt.ylabel('Total Business Value')

# Joining Designation Vs Total Business Value
plt.subplot(1, 3, 3)
```

```
sns.boxplot(data = df, x='Joining Designation', y='Total Business Value')
plt.title('Joining Designation Vs Total Business Value', fontweight='bold', color='red')
plt.xlabel('Joining Designation')
plt.ylabel('Total Business Value')

plt.tight_layout()
```



Gender Vs Total Business Value: Male Drivers: Larger in number but with slightly lower average and median business value, and higher negative outliers (up to -₹6M).

Female Drivers: Fewer in number but outperform males on average, median, top quartile, and maximum business value (up to ₹33.75M).

Grade Vs Total Business Value: Grade 1: Lowest-performing drivers with the smallest average business value (₹356K) and limited contribution overall.

Grade 2: Slightly better performers than Grade 1, with moderate average earnings around ₹467K.

Grade 3: Mid-level performers showing steady improvement, averaging ₹596K in business value.

Grade 4: High-performing drivers with over ₹1.24M average value and strong median contribution.

Grade 5: Top-tier drivers generating the highest revenue (₹1.41M average), with a few achieving extremely high values.

Joining Designation Vs Total Business Value: Designation 1: Most common joining role with a strong average (₹617K) and high top-end contribution (up to ₹17.65M).

Designation 2: Moderately common role with slightly lower average (₹570K) but highest max value of ₹33.75M.

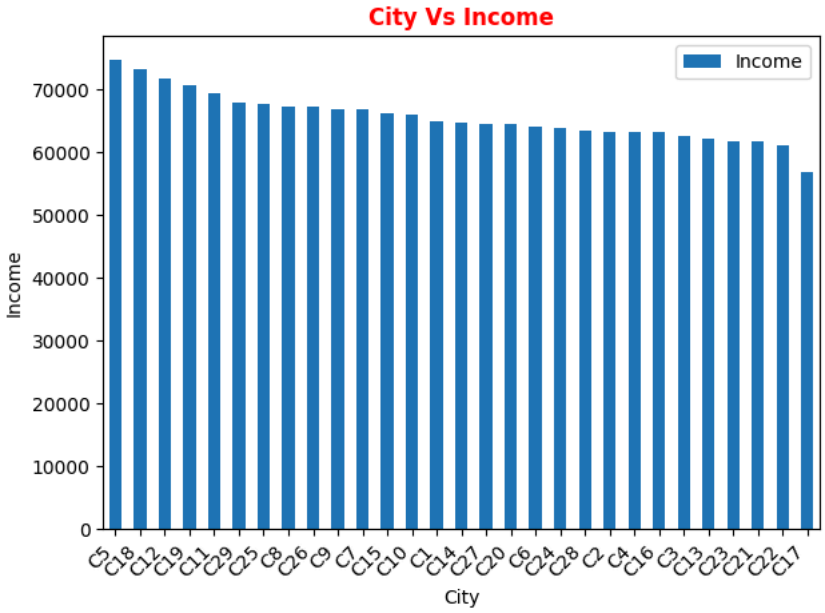
Designation 3: Lower average (₹382K) and median is zero, indicating many low or inactive performers.

Designation 4: High-performing group with ₹916K average and strong 75th percentile, though smaller in size.

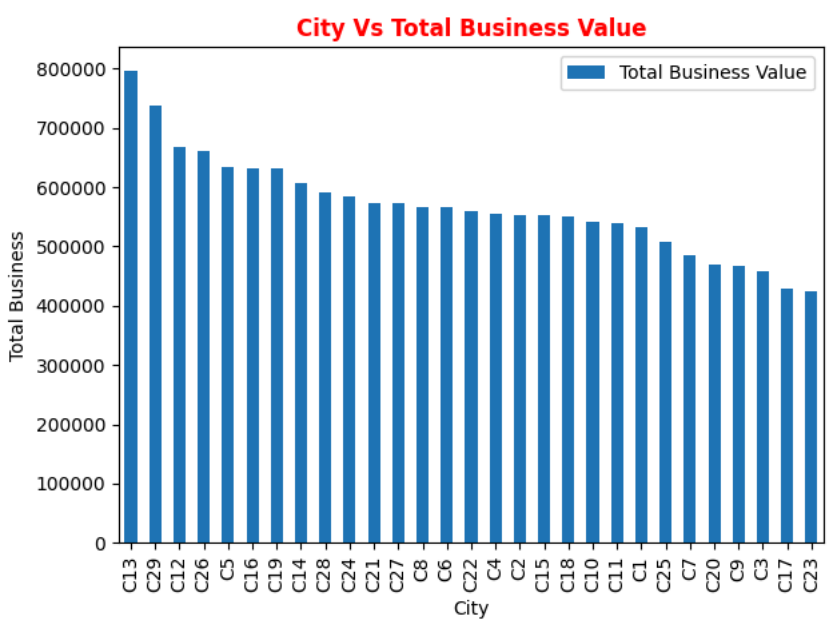
Designation 5: Smallest group with ₹447K average and limited activity (median = 0), but some high earners (up to ₹8.95M).

```
plt.figure(figsize=(6, 4))

# City Vs Income
city_vs_income = df.groupby('City')[['Income']].mean().round(2)
city_vs_income.sort_values(by='Income', ascending=False).plot(kind='bar')
plt.title('City Vs Income', fontweight='bold', color='red')
plt.xlabel('City')
plt.ylabel('Income')
plt.xticks(rotation=45, ha='right')

plt.tight_layout()
plt.show()
```

```
<Figure size 600x400 with 0 Axes>
```




```
# City vs Total Business Value
city_vs_total_business= df.groupby('City')[['Total Business Value']].mean().round(2)
city_vs_total_business.sort_values(by='Total Business Value', ascending=False).plot(kind='bar')
plt.title('City Vs Total Business Value', fontweight='bold', color='red')
plt.xlabel('City')
plt.ylabel('Total Business')


plt.tight_layout()
plt.show()
```




```
# Impact of city on Quarterly Rating, Grade , Income and Total Busines Value

city_wise_business = df.groupby('City')[['Quarterly Rating', 'Grade', 'Income', 'Total Business Value']].mean().round(2)

#Top 10 cities with 'Quarterly Rating', 'Grade', 'Income', 'Total Business Value'
city_wise_business.sort_values(by=['Total Business Value', 'Income','Quarterly Rating'], ascending=False)[:10]
```

City	Quarterly Rating	Grade	Income	Total Business Value
C13	2.03	2.46	62173.03	796263.08
C29	2.18	2.49	67847.28	736637.51
C12	2.01	2.59	71676.98	667282.31
C26	2.13	2.35	67186.54	661837.45
C5	2.05	2.29	74745.91	634855.98
C16	2.10	2.18	63141.39	632585.71
C19	2.05	2.46	70735.86	630978.15
C14	2.08	2.11	64680.24	607931.64
C28	2.01	2.24	63482.83	591406.78
C24	2.15	2.12	63774.36	584712.43

Quarterly Rating: Highest Average Rating: City C29 (2.18), followed by C24 and C26 (2.15 and 2.13 respectively). Lowest Average Rating: City C17 (1.77), C25 (1.86), and C20/C9 (1.85). Cities like C13, C29, C12, and C26 show relatively better performance based on Quarterly Ratings.

Grade: Highest Grade: City C12 (2.59), followed by C13 (2.46), C19 (2.46), and C29 (2.49). Lowest Grade: City C3 (1.95) and C23 (1.97). Generally, cities with higher grades also tend to have higher Quarterly Ratings (e.g., C29, C13).

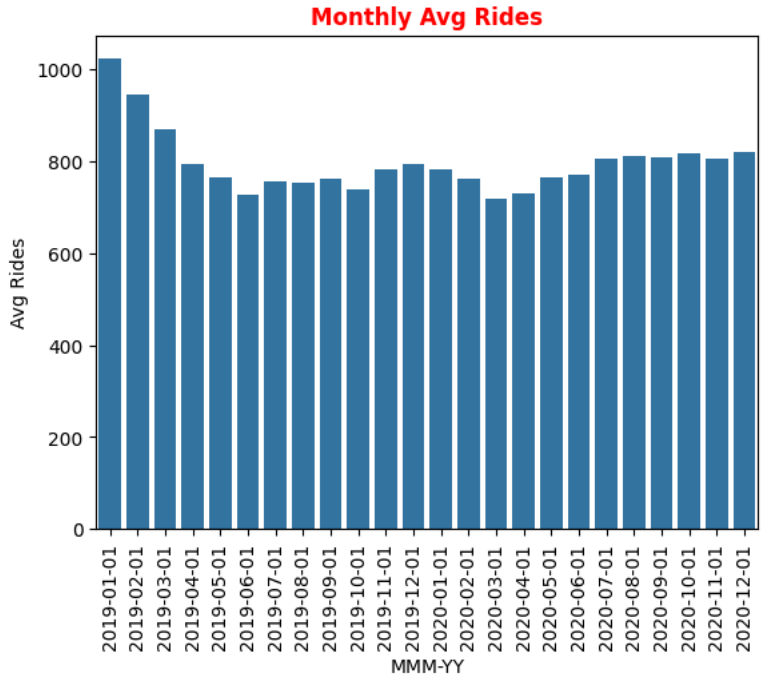
Income: Highest Income: City C5 (₹74,745.91), C18 (₹73,213.17), and C12 (₹71,676.98). Lowest Income: City C17 (₹56,845.65) and C16 (₹63,141.39). Income doesn't always correlate linearly with Quarterly Rating or Grade. For example, C18 has high income but only moderate rating (2.08).

Total Business Value: Highest Business Value: City C13 (₹796,263.08), C29 (₹736,637.51), C12 (₹667,282.31). Lowest Business Value: City C23 (₹423,986.56) and C17 (₹429,160.20).

There's a general trend where cities with higher Quarterly Ratings and Grades also yield higher business value, though not perfectly aligned.

```
# Monthly Avg Rides
Avg_rides_monthly= df.groupby('MMM-YY')['Driver_ID'].count()
sns.barplot(x=Avg_rides_monthly.index, y=Avg_rides_monthly.values)
plt.title('Monthly Avg Rides', fontweight='bold', color='red')
plt.xlabel('MMM-YY')
plt.xticks(rotation=90)
plt.ylabel('Avg Rides')
```

```
Text(0, 0.5, 'Avg Rides')
```



Rides per Months:

2019: High Rides: January 2019 had the highest ride count overall (1016), signaling a strong start to the year.

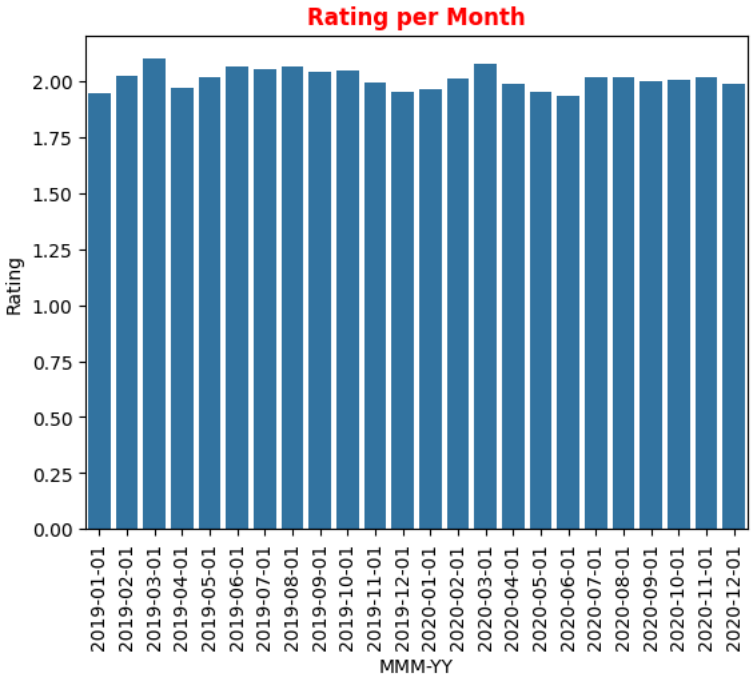
Low Rides: Activity declined from March (864) to June (716), showing a seasonal dip in the mid-year months.

2020: High Rides: The last quarter—October to December 2020—saw consistently high demand (~810 rides), with December (817) reaching the year's peak.

Low Rides: March to April saw a dip (708 to 722 rides) possibly due to early pandemic effects, with gradual recovery afterward.

```
# rRating per month
rating_per_month= df.groupby('MMM-YY')['Quarterly Rating'].mean()
sns.barplot(x=rating_per_month.index, y=rating_per_month.values)
plt.title('Rating per Month', fontweight='bold', color='red')
plt.xlabel('MMM-YY')
plt.xticks(rotation=90)
plt.ylabel('Rating')
```

```
Text(0, 0.5, 'Rating')
```

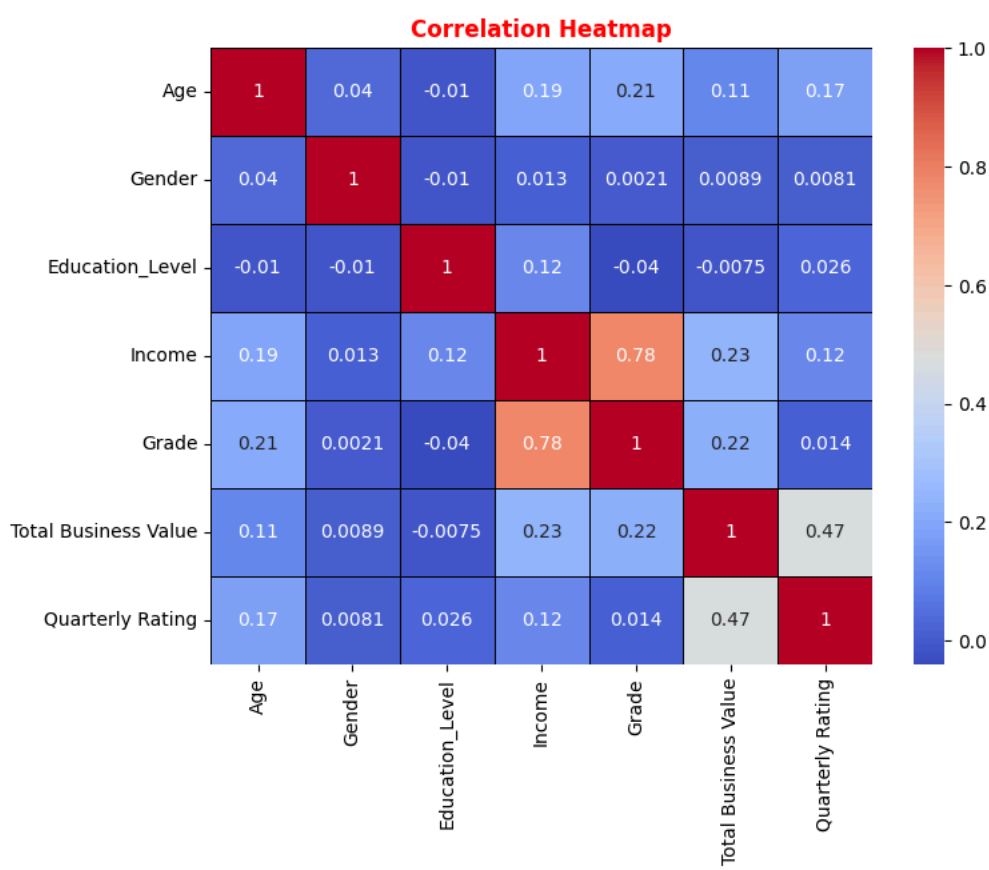


Monthly Rating:

In 2019, the quarterly rating peaked in March at 2.10 and was lowest in January and December, around 1.95–1.96, showing a slight dip toward the year-end. In 2020, the rating followed a similar pattern, peaking in March at 2.09, dropping to its lowest in June at 1.94, likely due to mid-year disruptions, before gradually recovering from July to November and dipping slightly again in December.

Correlation between features:

```
# Heat Map for corelation
plt.figure(figsize=(8, 6))
corr_col= df.drop(columns=['MMM-YY','Driver_ID','City','Dateofjoining','LastWorkingDate', 'Joining Designation'])
corr_mat= corr_col.corr()
sns.heatmap(corr_mat, annot=True, cmap='coolwarm', linewidths=0.5, linecolor='black')
plt.title("Correlation Heatmap", fontweight='bold', color='red')
plt.show()
```



Correlation Summary:

- Income is strongly correlated with Grade (0.78), suggesting that higher grades are associated with higher income.
- Total Business Value has a moderate correlation with Quarterly Rating (0.47) and Income (0.23), indicating that better-rated drivers and those with higher incomes tend to generate more business.
- Age shows weak positive correlations with Income (0.19), Grade (0.21),Quarterly Rating (0.17),Total Business Value (0.11).
- Education Level has a mild positive correlation with Income (0.12), but a slight negative correlation with Grade (−0.04), indicating no strong linear relationship.
- Gender is weakly correlated with all variables (correlation values near zero), suggesting little to no linear relationship.

Start coding or generate with AI.

Missing Value Treatment:

The features Age and Gender have 61 and 52 missing values respectively, while LastWorkingDate has 17,488 missing values. I will first treat the missing values in Age and Gender, and deal with LastWorkingDate later since it has a large proportion of missing data.

```
# Since most of the ages fall between 28 to 36, I will use the median to
replace missing values in the Age column. Using the mean could result in
fractional values, which may not be appropriate for representing age.
df['Age']= df['Age'].fillna(df['Age'].median())

# Gender
df['Gender']= df['Gender'].fillna(df['Gender'].mode()[0])
```

Outlier Treatment:

```
# Detection of skewness in the data.
df[["Age","Income","Total Business Value"]].skew()
```

	0
Age	0.437843
Income	0.705985
Total Business Value	6.970166

dtype: float64

```
# Count of Outliers in Age

Q1_age = df['Age'].quantile(0.25)
Q3_age = df['Age'].quantile(0.75)
IQR_age = Q3_age - Q1_age
lower_bound_age = Q1_age - 1.5 * IQR_age
upper_bound_age = Q3_age + 1.5 * IQR_age
outliers_age_count = df[(df['Age'] < lower_bound_age) | (df['Age'] > upper_bound_age)].shape[0]
print(f"Number of outliers in Age: {outliers_age_count}")
```

```
Number of outliers in Age: 78
```

```
# Outliers for Income
Q1_income = df['Income'].quantile(0.25)
Q3_income = df['Income'].quantile(0.75)
IQR_income = Q3_income - Q1_income
lower_bound_income = Q1_income - 1.5 * IQR_income
upper_bound_income = Q3_income + 1.5 * IQR_income
outliers_income_count = df[(df['Income'] < lower_bound_income) | (df['Income'] > upper_bound_income)].shape[0]
print(f"Number of outliers in Income: {outliers_income_count}")
```

```
Number of outliers in Income: 188
```

```
# Outlier count for Total buisness vlue
Q1_total_business_value = df['Total Business Value'].quantile(0.25)
Q3_total_business_value = df['Total Business Value'].quantile(0.75)
IQR_total_business_value = Q3_total_business_value - Q1_total_business_value
lower_bound_total_business_value = Q1_total_business_value - 1.5 * IQR_total_business_value
upper_bound_total_business_value = Q3_total_business_value + 1.5 * IQR_total_business_value
outliers_total_business_value_count = df[(df['Total Business Value'] < lower_bound_total_business_value) | (df['Total Busi
print(f"Number of outliers in Total Business Value: {outliers_total_business_value_count}")
```

```
Number of outliers in Total Business Value: 1371
```

Outlier treatment of feature Age, Income and Total Business value:

```
# Calculate IQR for Age
Q1 = df['Age'].quantile(0.25)
Q3 = df['Age'].quantile(0.75)
IQR = Q3 - Q1

# Define bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Cap outliers
df['Age'] = np.where(df['Age'] < lower_bound, lower_bound, df['Age'])
df['Age'] = np.where(df['Age'] > upper_bound, upper_bound, df['Age'])
```

```
# Calculate IQR for Income
Q1_income = df['Income'].quantile(0.25)
```

```
Q3_income = df['Income'].quantile(0.75)
IQR_income = Q3_income - Q1_income

# Define bounds for Income
lower_bound_income = Q1_income - 1.5 * IQR_income
upper_bound_income = Q3_income + 1.5 * IQR_income

# Cap outliers in Income
df['Income'] = np.where(df['Income'] < lower_bound_income, lower_bound_income, df['Income'])
df['Income'] = np.where(df['Income'] > upper_bound_income, upper_bound_income, df['Income'])
```

```
# Log transformation for Total Business because a large data points are outliers inputing all can lead to data loss and th
df['Total Business Value'] = np.log1p(df['Total Business Value'])
```

```
# Skweness after the outlier removal
df[["Age","Income","Total Business Value"]].skew()
```

	0
Age	0.412160
Income	0.622243
Total Business Value	-0.612929

dtype: float64

Before log transformtaion skew value was 6.97 which indiacated highly right-skewed data (long tail on the right). after log transformation data is slightly left-skewed, but much closer to symmetric.

Negative Total Business Value entries have resulted in NaN values. These can adversely affect the machine learning model, so I will drop these NaN records.

```
# Droping Nan Value of Total Business Value
df.dropna(subset=["Total Business Value"], inplace=True)
```

```
df["Total Business Value"].isna().sum()
```

```
np.int64(0)
```

Feature Enginnering:

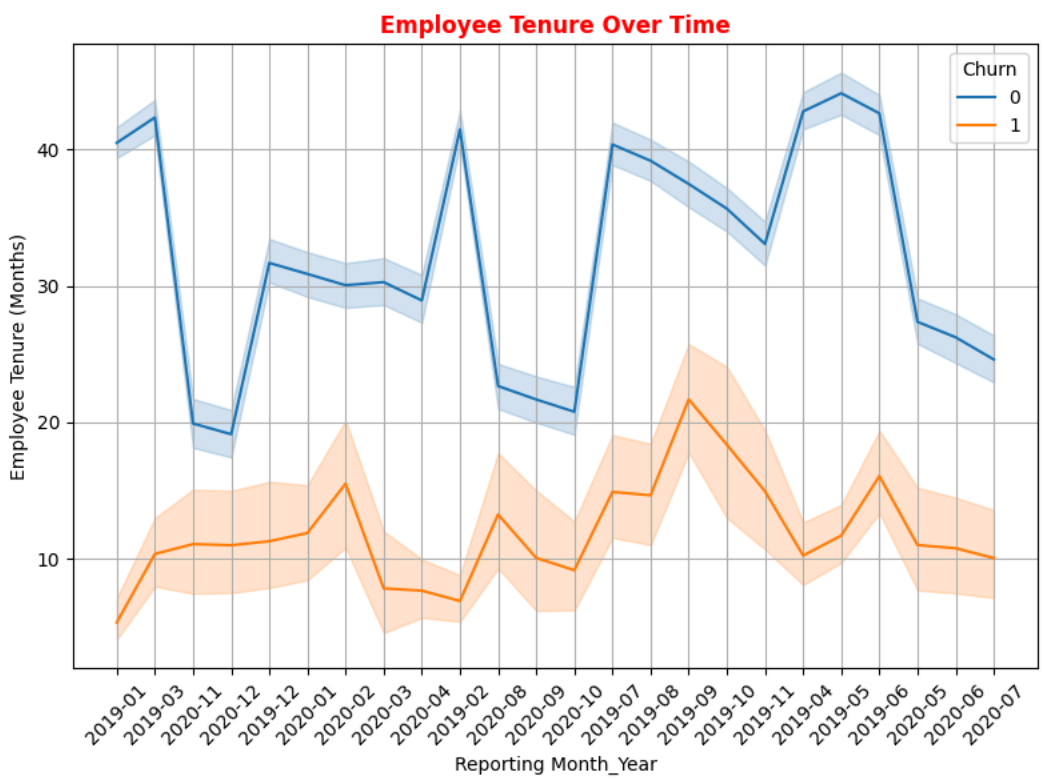
```
# Extract reporting month and reporting year from MMM-YY
df['Reporting_Month'] = df['MMM-YY'].dt.to_period('M')
df['Reporting_Year'] = df['MMM-YY'].dt.to_period('Y')
df['Reporting_Month_Year'] = df['MMM-YY'].dt.to_period('M')
df['Reporting_Quarter'] = df['MMM-YY'].dt.to_period('Q')
```

Creating a new feature 'Churn' from 'LastWorkingDay': drivers who have left have a recorded last working day, while those who stayed do not.

```
# Drive's Churn, 1:Churn, 0:No Churn
df['Churn'] = df['LastWorkingDate'].apply(lambda x: 0 if pd.isna(x) else 1)
```

```
# Categorize the 'Age' feature into bins such as: 'Young' (18–30), 'Middle-aged' (31–50), and 'Senior' (51+).

df['Age_Category'] = pd.cut(df['Age'], bins=[18, 30, 50, float('inf')],
                            labels=['Young', 'Middle-aged', 'Senior'])
```

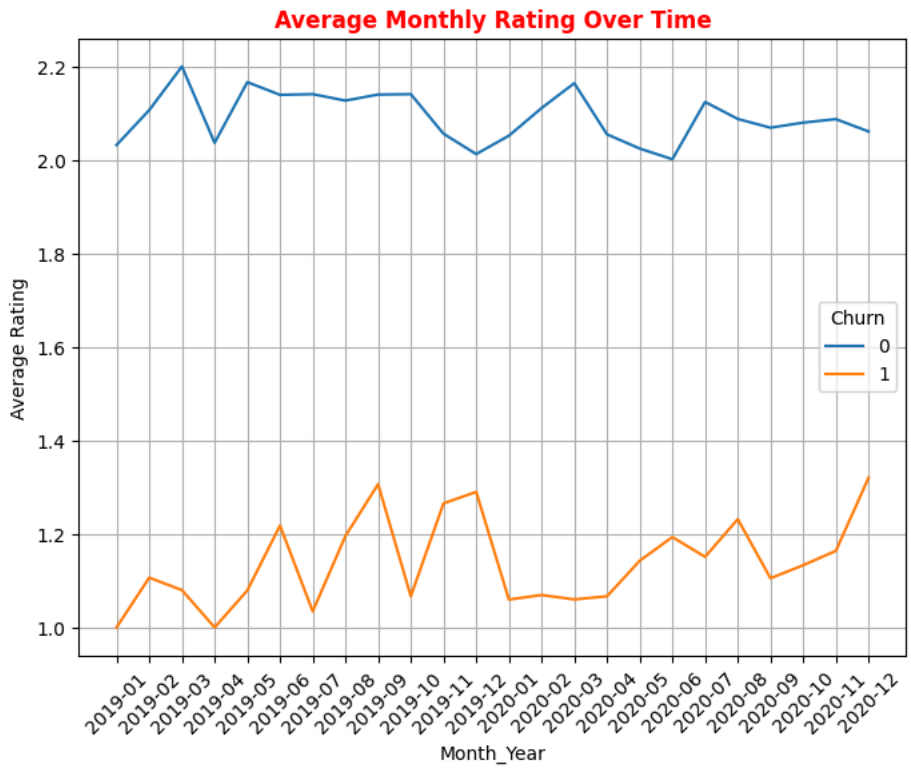
```
# Driver's tenure
def tenure(X):
    if pd.isna(X['LastWorkingDate']):
        # For drivers who have not churned, calculate tenure up to the latest reporting date
        return df['MMM-YY'].max() - X['Dateofjoining']
    else:
        # For drivers who have churned, calculate tenure from joining date to last working date
        return X['LastWorkingDate'] - X['Dateofjoining']



df['Employee_Tenure'] = df.apply(tenure, axis=1)
df['Employee_Tenure_Months'] = df['Employee_Tenure'].dt.days/30


# Emloyee_Tenure_Months Vs Churn
plt.figure(figsize=(8, 6))
sns.lineplot(data=df, x=df['Reporting_Month_Year'].astype(str), y='Employee_Tenure_Months', hue='Churn')
plt.title('Employee Tenure Over Time', fontweight='bold', color='red')
plt.xlabel('Reporting Month_Year')
plt.xticks(rotation=45)
plt.ylabel('Employee Tenure (Months)')
plt.grid(True)
plt.tight_layout()
```



We can observe that drivers who have churned have a much lower tenure period compared to those who stayed. Churned drivers have a mean tenure of around 12 months, while non-churned drivers have a mean tenure of approximately 32 months.

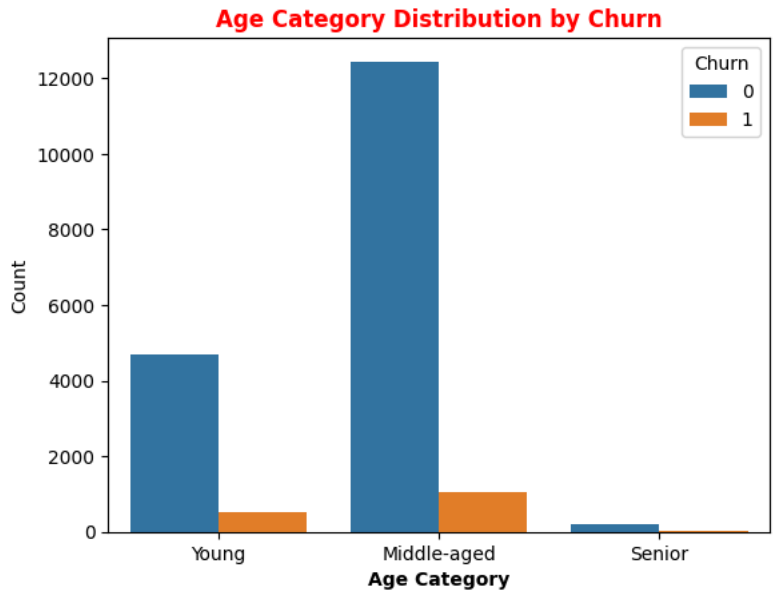
```
# Rating Change over time
rating_over_time = df.groupby(['Reporting_Month_Year', 'Churn'])['Quarterly Rating'].mean().reset_index()

# Converting 'Reporting_Month_Year' to string for plotting compatibility
rating_over_time['Reporting_Month_Year'] = rating_over_time['Reporting_Month_Year'].astype(str)

# Plot between Reporting Month_Year and Quarterly Rating
plt.figure(figsize=(8,6))
sns.lineplot(data=rating_over_time, x='Reporting_Month_Year', y='Quarterly Rating',hue='Churn')
plt.title("Average Monthly Rating Over Time", fontweight='bold' , color='red')
```

```
plt.ylabel("Average Rating")
plt.xticks(rotation=45)
plt.xlabel("Month_Year")
plt.grid(True)
plt.show()
```



We can observe that drivers who churned have an average monthly rating around 1, while drivers who did not churn have an average rating around 2.

```
# Plot Age category
sns.countplot(data=df, x="Age_Category", hue="Churn")
plt.title("Age Category Distribution by Churn",fontweight="bold", color="red")
plt.xlabel("Age Category", fontweight="bold")
plt.ylabel("Count")
plt.show()
```




```
# Churn percentage per Age Category
df.groupby('Age_Category')['Churn'].value_counts(normalize=True).reset_index()
```

	Age_Category	Churn	proportion
0	Young	0	0.898683
1	Young	1	0.101317
2	Middle-aged	0	0.922889
3	Middle-aged	1	0.077111
4	Senior	0	0.938326
5	Senior	1	0.061674

Among Young drivers, 10.13% have churned, while 89.87% have not. Among Middle-aged drivers, the churn rate is slightly lower at 7.71%, with 92.29% staying. Among Senior drivers, churn is the lowest at 6.17%, with 93.83% retained.

This suggests that younger drivers are more likely to churn, while senior drivers show the highest retention.

Performance Feedback:

```
# df 2019
df_2019 = df[df['Reporting_Year'] == '2019']
# df 2020
df_2020 = df[df['Reporting_Year'] == '2020']

# Improvement in Qutarly Rating by city
df_city = df_2020.merge(df_2019, on='City', suffixes=('_2020', '_2019'))
df_city = df_city.groupby('City')[['Quarterly Rating_2020', 'Quarterly Rating_2019']].mean().reset_index()
df_city['Rating_diff']=df_city['Quarterly Rating_2020'] - df_city['Quarterly Rating_2019']
df_city.sort_values(by='Rating_diff', ascending=False)[:5]
```

	City	Quarterly Rating_2020	Quarterly Rating_2019	Rating_diff
21	C29	2.269903	2.068783	0.201120
6	C15	2.061381	1.918033	0.143348
16	C24	2.236162	2.092814	0.143348
24	C5	2.097143	1.990066	0.107077
27	C8	2.098404	2.014970	0.083434

Top 5 Cities:

City C29 recorded the highest improvement, with its Quarterly Rating increasing by 0.201 points (from 2.068 in 2019 to 2.270 in 2020).

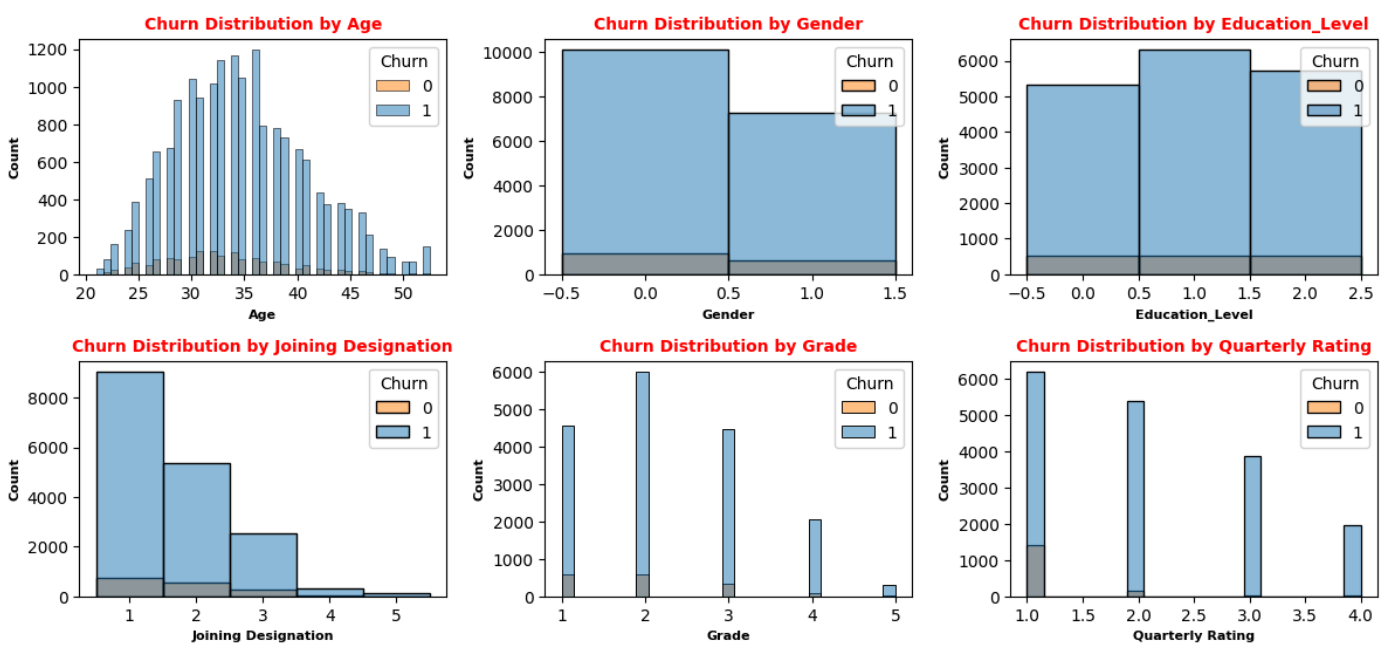
City C15 showed a significant gain of 0.143 points, rising from 1.918 to 2.061.

City C24 also improved by 0.143 points, moving up from 2.093 in 2019 to 2.236 in 2020.

City C5 experienced a moderate increase of 0.107 points, with its rating improving from 1.990 to 2.097.

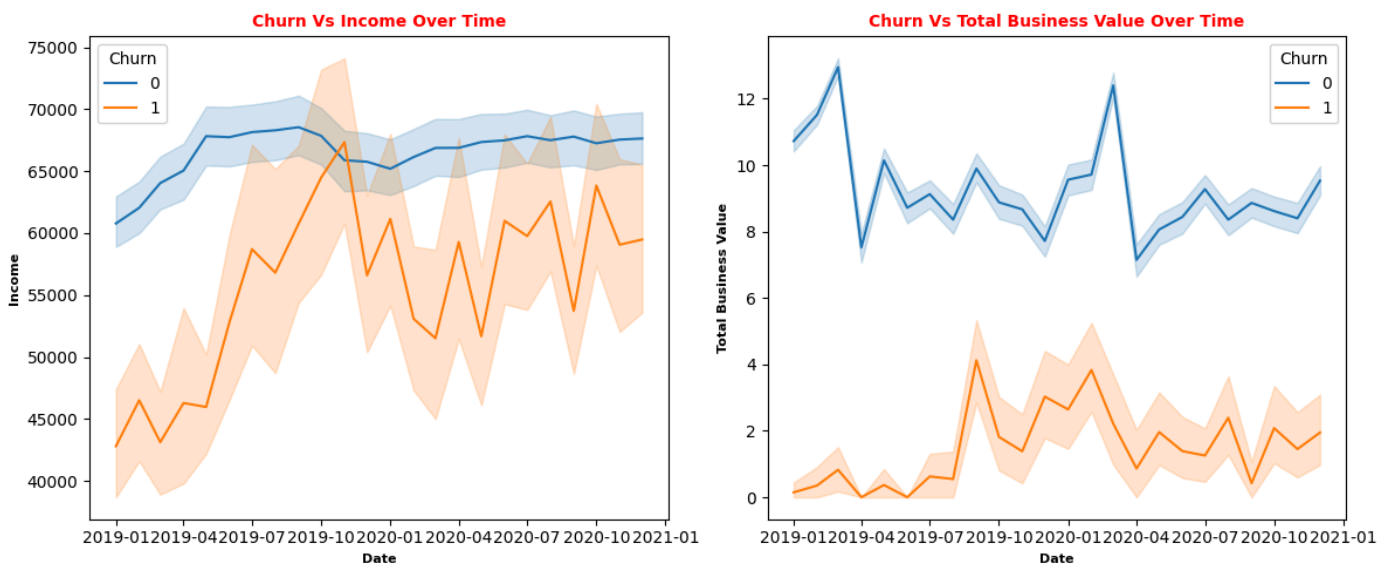
City C8 had a positive change of 0.083 points, increasing its rating from 2.015 to 2.098.

```
# Categorical Features Vs Churn

categorical_features = ['Age','Gender', 'Education_Level', 'Joining
Designation', 'Grade', 'Quarterly Rating']

# Calculate the number of rows needed for the subplots
n_rows = (len(categorical_features) + 1) // 2

plt.figure(figsize=(12, 8))
for i, feature in enumerate(categorical_features):
    plt.subplot(n_rows, 3, i + 1)
    sns.histplot(data=df, x=feature, hue='Churn')
    plt.title(f'Churn Distribution by {feature}', fontsize=10,
    fontweight='bold', color="red")
    plt.xlabel(feature, fontsize=8, fontweight='bold')
    plt.ylabel('Count', fontsize=8, fontweight='bold')

    plt.legend(title='Churn', loc='upper right', labels=[0,1])


plt.tight_layout()
plt.show()
```



- The churn rate is almost the same for both genders, indicating that gender does not have a significant impact on churn in this dataset.
- Drivers with Education Level 1 show the lowest churn rate. However, overall churn across all education levels are same.
- This trend indicates that employees who join at senior positions are less likely to leave the organization.
- There is a clear decreasing trend in churn rate with higher Grades.Grade 1 has the highest churn, while Grade 4 has the lowest churn(3.29%).
- There is a strong inverse relationship between Quarterly Rating and Churn Rate.Lower-rated drivers are far more likely to churn.

```
# Churn Vs Numeric Features
numerical_features = ['Income', 'Total Business Value']

plt.figure(figsize=(12, 5))
plt.subplot(1,2,1)
sns.lineplot(data=df, x='MMM-YY', y='Income', hue='Churn')
plt.title('Churn Vs Income Over Time', fontsize=10, fontweight='bold', color="red")
plt.xlabel('Date', fontsize=8, fontweight='bold')
plt.ylabel('Income', fontsize=8, fontweight='bold')


plt.subplot(1,2,2)
sns.lineplot(data=df, x='MMM-YY', y='Total Business Value', hue='Churn')
plt.title('Churn Vs Total Business Value Over Time', fontsize=10, fontweight='bold', color="red")
plt.xlabel('Date', fontsize=8, fontweight='bold')
plt.ylabel('Total Business Value', fontsize=8, fontweight='bold')


plt.tight_layout()
plt.show()
```



Churned drivers earn significantly less on average than those who stay. Income has a strong inverse relationship with churn — as income increases, churn drops.

There is a clear inverse relationship: higher total business value is associated with lower churn rates. Retained drivers are more engaged and contribute consistently more business value over time.

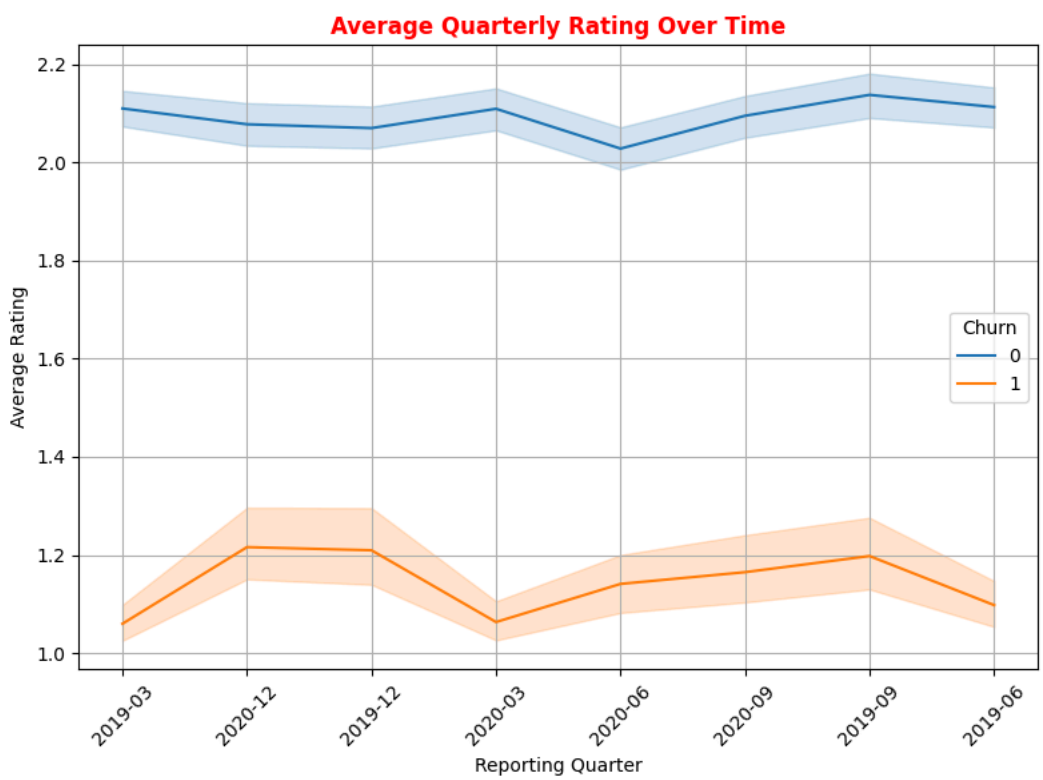
```
# Drivers Quarterly Avg Rating
df['Average_Quarterly_Rating'] = df.groupby(['Driver_ID','Reporting_Quarter'])
['Quarterly Rating'].transform('mean')
```

```
df.groupby('Churn')['Average_Quarterly_Rating'].describe()
```

Churn	count	mean	std	min	25%	50%	75%	max
0	17370.0	2.092458	1.009850	1.0	1.0	2.0	3.0	4.0
1	1585.0	1.141956	0.438849	1.0	1.0	1.0	1.0	4.0

Drivers who churned had significantly lower quarterly ratings (mean: 1.14) compared to those who didn't churn (mean: 2.09). Most churned drivers had ratings around 1.0, suggesting low ratings are strongly associated with higher churn likelihood.

```
df['Quarterly_Rating_Change'] = df.groupby('Driver_ID')['Quarterly Rating'].diff()



# first entry does not have previous value to campare with so it created
df['Quarterly_Rating_Change'].fillna(0, inplace=True)


# Plto between Average Quarterly rating and Reporting_Quarter
plt.figure(figsize=(8, 6))

# Convert Reporting_Quarter to string for plotting

df['Reporting_Quarter_str'] = df['Reporting_Quarter'].dt.strftime('%Y-%m')
sns.lineplot(data=df, x='Reporting_Quarter_str', y='Average_Quarterly_Rating',hue='Churn')
plt.title("Average Quarterly Rating Over Time", fontweight='bold' , color='red')
plt.ylabel("Average Rating")
plt.xticks(rotation=45)
plt.xlabel("Reporting Quarter")
plt.grid(True)
plt.tight_layout()
plt.show()
```



Similar to monthly rating changes, the quarterly rating also indicates that drivers who churned had an average rating around 1, while those who did not churn had an average rating around 2.

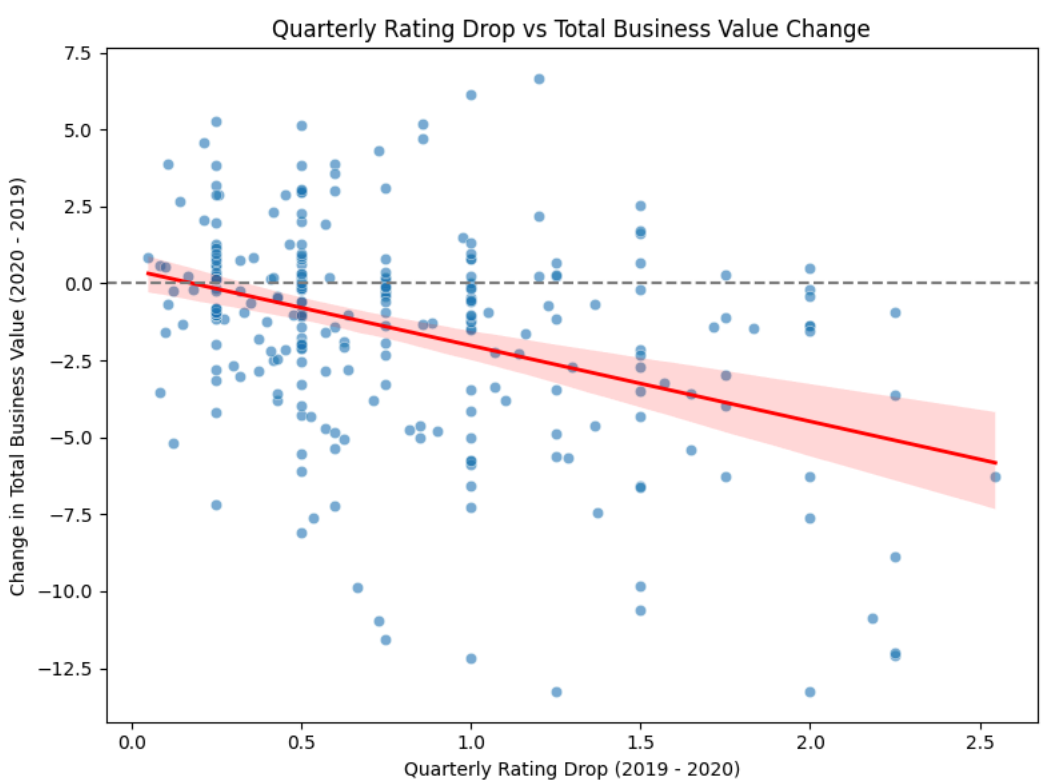
Start coding or generate with AI.

```
# Step 1: Quarterly Rating for 2019 and 2020
Q_rating_2019 = df[df['Reporting_Year'] == pd.Period('2019')].groupby('Driver_ID')['Quarterly Rating'].mean().reset_index(
Q_rating_2020 = df[df['Reporting_Year'] == pd.Period('2020')].groupby('Driver_ID')['Quarterly Rating'].mean().reset_index(

# Step 2: Merge to calculate rating difference
df_rating = Q_rating_2020.merge(Q_rating_2019, on='Driver_ID')
```

```
df_rating['Rating_Diff'] = df_rating['Quarterly_Rating_2019'] - df_rating['Quarterly_Rating_2020']

# Step 3: Total Business Value for 2019 and 2020
tbv_2019 = df[df['Reporting_Year'] == pd.Period('2019')].groupby('Driver_ID')['Total Business Value'].mean().reset_index()
tbv_2020 = df[df['Reporting_Year'] == pd.Period('2020')].groupby('Driver_ID')['Total Business Value'].mean().reset_index()

# Step 4: Merge TBV and rating data
df_tb = tbv_2020.merge(tbv_2019, on='Driver_ID').merge(df_rating, on='Driver_ID')

# Step 5: Calculate TBV change
df_tb['TBV_Change'] = df_tb['TBV_2020'] - df_tb['TBV_2019']

# Step 6: Filter for drivers whose rating dropped
df_rating_drop = df_tb[df_tb['Rating_Diff'] > 0]

# Rading_Drop Vs TBV_change
df_rating_drop[['Driver_ID', 'Rating_Diff', 'TBV_2019', 'TBV_2020', 'TBV_Change']].describe()
```

	Driver_ID	Rating_Diff	TBV_2019	TBV_2020	TBV_Change
count	273.000000	273.000000	273.000000	273.000000	273.000000
mean	1486.868132	0.779183	11.171312	9.695719	-1.475593
std	821.490622	0.533689	2.596393	4.099535	3.404769
min	13.000000	0.050000	3.136139	0.000000	-13.247749
25%	790.000000	0.416667	9.860526	7.294364	-2.995502
50%	1486.000000	0.600000	11.897192	11.113942	-0.931099
75%	2204.000000	1.000000	13.223917	12.814541	0.256296
max	2773.000000	2.545455	14.380799	14.591645	6.644592

```
# Plot between Quarterly rating drop and Total Business value

plt.figure(figsize=(8, 6))
sns.scatterplot(data=df_rating_drop, x='Rating_Diff', y='TBV_Change', alpha=0.6)
sns.regplot(data=df_rating_drop, x='Rating_Diff', y='TBV_Change',
scatter=False, color='red', line_kws={'linewidth': 2})
plt.axhline(0, color='gray', linestyle='--')
plt.title('Quarterly Rating Drop vs Total Business Value Change')
plt.xlabel('Quarterly Rating Drop (2019 - 2020)')
plt.ylabel('Change in Total Business Value (2020 - 2019)')
plt.tight_layout()
plt.show()
```



Impact of Quarterly Rating Drop on Total Business Value (2019 → 2020):

273 drivers whose average quarterly rating dropped from 2019 to 2020.On average, their rating dropped by 0.78 points, and their Total Business Value decreased by ₹1.48 lakhs.

The median business value also declined, from ₹11.90L in 2019 to ₹11.11L in 2020, indicating a general downward trend in performance.

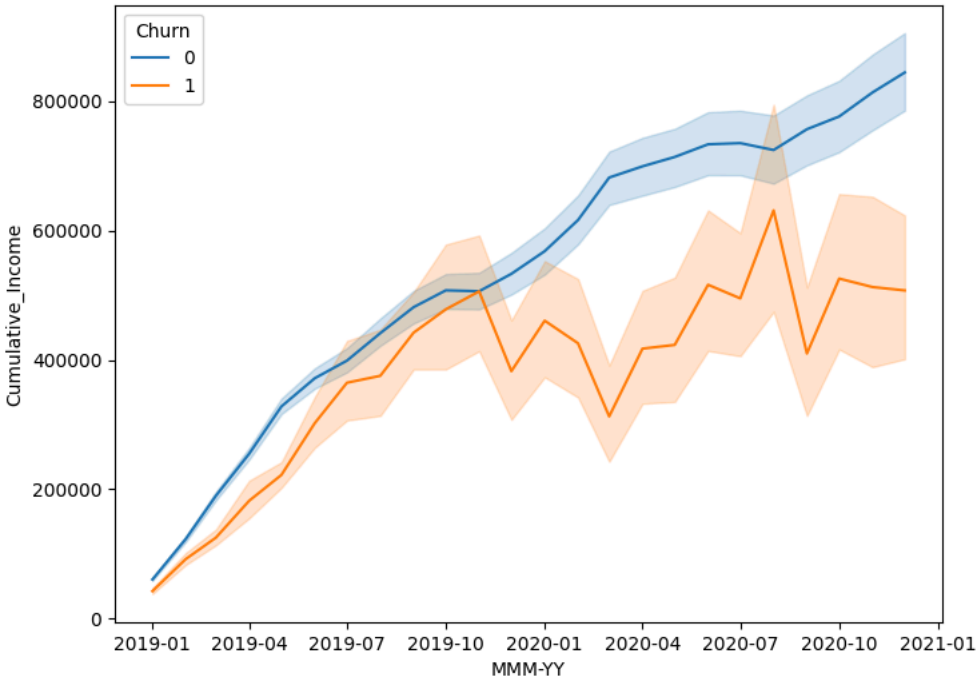
The maximum rating drop was 2.54 points, and the largest decline in Total Business Value was ₹13.25L, while some drivers even showed an increase of ₹6.64L in Total Business Value.

Aggregated Business Values:

Cumulative 'Total Business Value' for each driver up to each 'MMM-YY'. This could provide a better picture of their total contribution over time.

```
#  Cumulative Income
df = df.sort_values(by=['Driver_ID', 'MMM-YY'])
df['Cumulative_Income'] = df.groupby('Driver_ID')['Income'].cumsum()


# MMM-YY Vs Cumulative_Income
plt.figure(figsize=(8, 6))
sns.lineplot(data=df, x='MMM-YY', y='Cumulative_Income', hue='Churn')
```

```
<Axes: xlabel='MMM-YY', ylabel='Cumulative_Income'>
```




```
df.groupby('Churn')['Cumulative_Income'].describe()
```

Churn	count	mean	std	min	25%	50%	75%	max
0	17370.0	526961.327922	564747.398965	10747.0	128321.0	314740.5	720792.0	3512352.0
1	1585.0	376221.909148	387027.369794	10883.0	129141.0	243280.0	469980.0	2815380.0

Cumulative Income: Non-churned drivers have higher mean and median incomes, suggesting better earnings may contribute to driver retention.

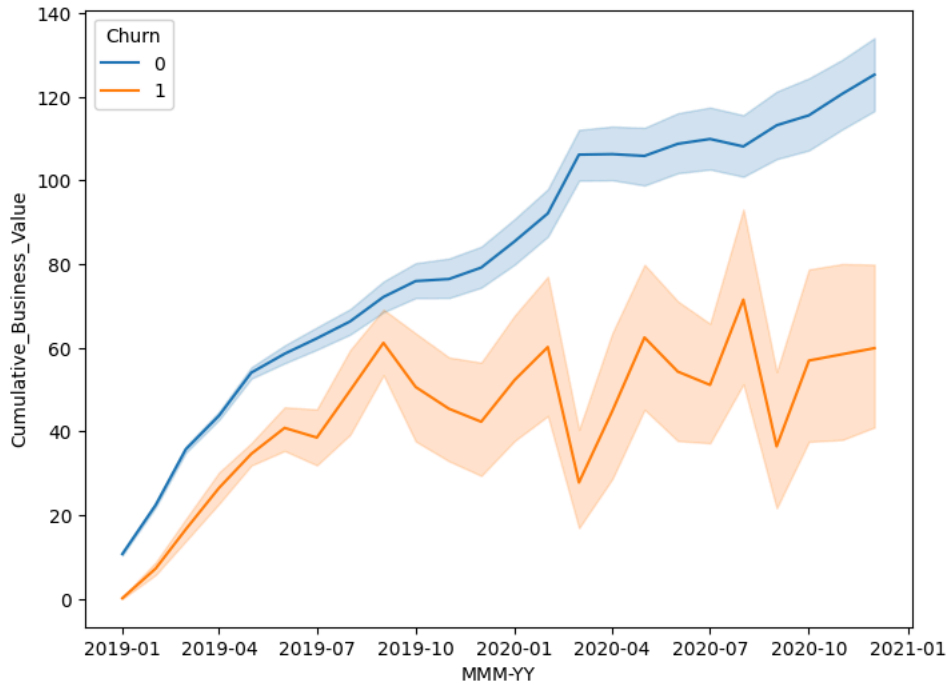
75% of non-churned drivers earn below ₹7.21 lakhs, while for churned drivers it's ₹4.70 lakhs, showing a wider and higher earning range among retained drivers.

The standard deviation is also higher for non-churned drivers, indicating more variability in their income, possibly due to longer tenure or higher engagement.

Minimum income values are similar, but non-churned drivers have a significantly higher maximum income.

```
# Cumulative Business Value
df = df.sort_values(by=['Driver_ID', 'MMM-YY'])
df['Cumulative_Business_Value'] = df.groupby('Driver_ID')['Total Business Value'].cumsum()


# Plot between MMM-YY and Cumulative_Business_Value
plt.figure(figsize=(8, 6))
sns.lineplot(data=df, x='MMM-YY', y='Cumulative_Business_Value', hue='Churn')
```

```
<Axes: xlabel='MMM-YY', ylabel='Cumulative_Business_Value'>
```




```
df.groupby('Churn')['Cumulative_Business_Value'].describe()
```

Churn	count	mean	std	min	25%	50%	75%	max
0	17370.0	80.181420	82.993379	0.0	13.03307	51.627298	125.069786	345.361110
1	1585.0	43.486636	58.885739	0.0	0.00000	24.112174	61.972070	326.858635

Cumulative_Business_Value: Non-churned drivers contribute almost double the business value on average compared to churned drivers.

Churned drivers have a higher proportion of zero or very low contributors.

The difference in median and upper percentiles confirms that drivers who stay longer contribute significantly more.

Droping some columns befor Data modeling:

- **Driver_ID**: This is likely a unique identifier for each driver. It doesn't inherently contain information that would be useful for predicting churn.
- **MMM-YY**: This column represents the reporting month and year.I've created features like 'Reporting_Month', 'Reporting_Year' and 'Reporting_Month_Year'. so droping it
- **Dateofjoining**: Similar to MMM-YY, this is a date column that I've used to create features like 'Employee_tenure'. The raw date might not be as informative as the derived features.
- **LastWorkingDate**: This column directly indicates churn (if not null). I've already created the 'Churn' column based on this, so LastWorkingDate should be dropped.
- **Employee_Tenure** : I have created Employee_Tenure_Months, so I am deleting Employee_Tenure because essentially both are the same, and I believe Employee_Tenure_Months is more compatible with the other columns.
- Dropping **Cumulative_Income** and **Cumulative_Total_Business_Value** because they are derived from the existing Income and Total_Business_Value columns. Since these features provide redundant information, retaining only the original columns helps reduce multicollinearity and simplifies the model.

```
# Droping columns
df = df.drop(columns=['Driver_ID', 'MMM-YY', 'Dateofjoining', 'LastWorkingDate','Reporting_Month_Year', 'Employee_Tenure',
```

Encoding Categorical Feature:

Check number of Unique value per feature

```
def unique_vals(col):

  if col.dtype == "object" or col.dtype== "category":

    print(f'{col.name}: {col.nunique()}')

df.apply(lambda col: unique_vals(col))
```

```
Gender: 2
City: 29
Education_Level: 3
Joining Designation: 5
Age_Category: 3
```

	0
Age	None
Gender	None
City	None
Education_Level	None
Income	None
Joining Designation	None
Grade	None
Total Business Value	None
Quarterly Rating	None
Reporting_Month	None
Reporting_Year	None
Reporting_Quarter	None
Churn	None
Age_Category	None
Employee_Tenure_Months	None
Average_Quarterly_Rating	None
Quarterly_Rating_Change	None

dtype: object

I will use binary encoding for features with 2 or less. For features < 6 unique values I will use OneHot encoding Rest of the categorical features will be Target encoded

```
# label encoding for Grade
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
le = LabelEncoder()
df["Grade"] = le.fit_transform(df["Grade"])

df['Grade'].value_counts()
```

Grade	count
1	6582
0	5163
2	4782
3	2126
4	302

dtype: int64

```
# Label encoding on Gender
le = LabelEncoder()
df["Gender"] = le.fit_transform(df["Gender"])
```

```
le=LabelEncoder()
df["Education_Level"]=le.fit_transform(df["Education_Level"])
```

```
# OneHot Encoding on Age_category, Joining Designation and Grade

df = pd.get_dummies(df, columns = ["Age_Category", "Joining Designation"], drop_first = False)
```

Conversion of Period data type to integer.

```
df['Reporting_Month'] = df['Reporting_Month'].dt.month
df = pd.get_dummies(df, columns=['Reporting_Month'], prefix='Reporting_Month')
```

```
df['Reporting_Year'] = df['Reporting_Year'].dt.month
df = pd.get_dummies(df, columns=['Reporting_Year'], prefix='Reporting_Year')
```

```
df['Reporting_Quarter'] = df['Reporting_Quarter'].dt.month
df = pd.get_dummies(df, columns=['Reporting_Quarter'],
prefix='Reporting_Quarter')
```

```
# Conver 0 and 1
df[["Age_Category_Young", "Age_Category_Middle-aged", "Age_Category_Senior", "Joining Designation_1", "Joining Designation_
```

```
df[['Reporting_Month_1','Reporting_Month_2','Reporting_Month_3', 'Reporting_Month_4', 'Reporting_Month_5', 'Reporting_Montl
'Reporting_Year_12','Reporting_Quarter_3', 'Reporting_Quarter_6', 'Reporting_Quarter_9', 'Reporting_Quarter_12']] = df[['R
'Reporting_Year_12','Reporting_Quarter_3', 'Reporting_Quarter_6', 'Reporting_Quarter_9', 'Reporting_Quarter_12']].astype(i
```

```
# Column name Change
df.rename(columns={'Reporting_Year_12': 'Reporting_Year', 'Reporting_Quarter_3': 'Reporting_Quarter_1', 'Reporting_Quarter_
```

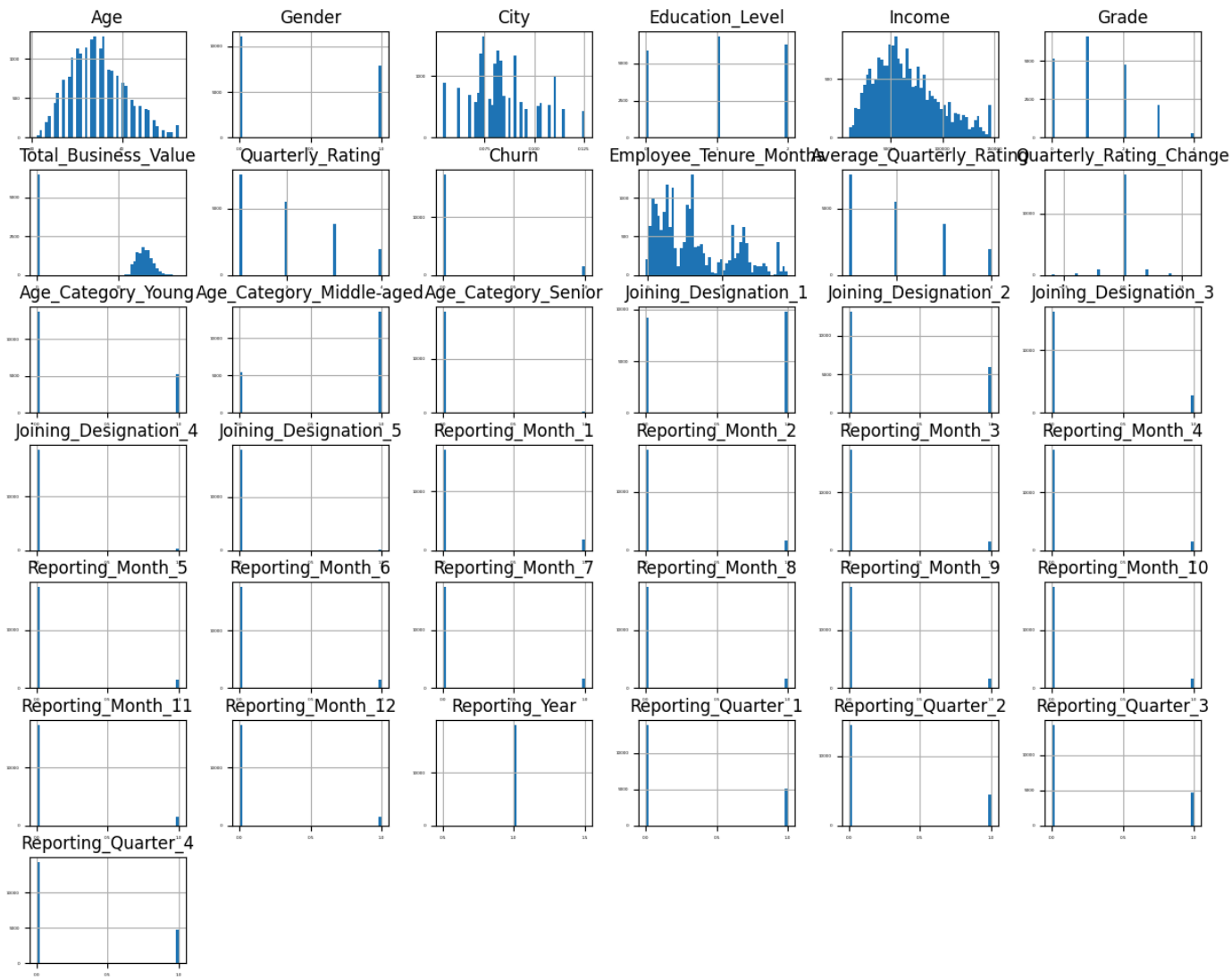
```
pip install category_encoders
```

```
Requirement already satisfied: category_encoders in /usr/local/lib/python3.11/dist-packages (2.8.1)
Requirement already satisfied: numpy>=1.14.0 in /usr/local/lib/python3.11/dist-packages (from category_encoders) (2.0.2)
Requirement already satisfied: pandas>=1.0.5 in /usr/local/lib/python3.11/dist-packages (from category_encoders) (2.2.2)
Requirement already satisfied: patsy>=0.5.1 in /usr/local/lib/python3.11/dist-packages (from category_encoders) (1.0.1)
Requirement already satisfied: scikit-learn>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from category_encoders) (1.6.1)
Requirement already satisfied: scipy>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from category_encoders) (1.16.1)
Requirement already satisfied: statsmodels>=0.9.0 in /usr/local/lib/python3.11/dist-packages (from category_encoders) (0.14.5)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.0.5->category_encoders)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.0.5->category_encoders) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas>=1.0.5->category_encoders) (2025.2
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn>=1.6.0->category_encoders) (1
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn>=1.6.0->category_encod
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.11/dist-packages (from statsmodels>=0.9.0->category_encoders) (
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.8.2->pandas>=1.0.5->category
```

```
# Target encoding on feature City
from category_encoders import TargetEncoder
encoder = TargetEncoder()
df['City'] = encoder.fit_transform(df['City'], df['Churn'])
```

```
df.columns = df.columns.str.replace(' ', '_')
```

```
# Plot all features
df.hist(figsize=(15, 12), bins=50, xlabelsize=3, ylabelsize=3)
plt.show()
```



Why are some features skewed?

- Age_Category_Young

Represents whether the person is in the "Young" age group.

About 27.6% of data falls into this group.

Since most values are 0, it's a left-skewed feature.

- Age_Category_Middle-aged

Indicates if the person is "Middle-aged".

Majority class with 71.1% of entries.

Has many 1s → right-skewed feature.

- Age_Category_Senior

Flags individuals in the "Senior" age group.

Very rare (only ~1.2% of data).

Most values are 0 → highly left-skewed feature.

- Joining_Designation_1

Marks if the joining designation was level 1.

Common category (~51.5% of drivers).

Many 1s → right-skewed feature.

- Joining_Designation_2

Represents designation level 2 at joining.

Occurs in 31.1% of cases.

More 0s than 1s → moderately right-skewed.

- Joining_Designation_3

Shows if the driver joined at designation level 3.

Around 14.9% occurrence.

Mostly 0s → mildly left-skewed.

- Joining_Designation_4

Designation level 4 at joining.

Rare category (only ~1.8%).

Most values are 0 → left-skewed.

- Joining_Designation_5

Represents the rarest joining designation level.

Just 0.68% of the data.

Almost entirely 0s → highly left-skewed.

Start coding or generate with AI.

Model Building:

```
# Train test data split
from sklearn.model_selection import train_test_split
X = df.drop(columns=['Churn'])
y = df['Churn']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
print( "X_train Dataset: ", X_train.shape)
print( "X_test Dataset: ", X_test.shape)
print( "y_train Dataset: ", y_train.shape)
print( "y_test Dataset: ", y_test.shape)
```

```
X_train Dataset:  (15164, 36)
X_test Dataset:  (3791, 36)
y_train Dataset:  (15164,)
y_test Dataset:  (3791,)
```

We can observe that the training dataset has 15,164 rows and 36 columns (features), while the test dataset has 3,791 rows and 1 column (target).

```
# Class distibution of target valriable
y_train.value_counts()
```

Churn	count
0	13882
1	1282

dtype: int64

Here we can see that the class distribution is imbalanced; hence, I will apply the SMOTE technique to balance the classes.

```
# SMOTE application on target Feature Y
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_train, y_train = smote.fit_resample(X_train, y_train)
```

```
y_train.value_counts()
```

Churn	count
0	13882
1	13882

dtype: int64

Now classes of training dataset are balanced.

Ensemble Learning: Random Forest Classifier

Why I am using RandomForestClassifier over BaggingClassifier?

Random Forest is a modeling technique based on bagging and decision trees. It introduces additional randomness by selecting a random subset of features (columns) and samples (rows) for each base learner. The base learners in Random Forest are decision trees, and they are trivially parallelizable since each tree is trained independently. I prefer using Random Forest over plain bagging because the additional randomness helps reduce variance and prevents overfitting.

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(random_state=42)
rf.fit(X_train, y_train)
```

```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

```
rf.score(X_train, y_train)
```

```
0.999963982135139
```

Training score of 0.99 suggesting overfitting.

```
from sklearn.model_selection import RandomizedSearchCV

# Model training
param_dist = {
    'n_estimators': [100, 200, 300, 400],
    'max_depth': [5, 10, 20, None],
    'min_samples_split': [2, 5, 10,15,20],
    'min_samples_leaf': [1, 2, 4],
    'bootstrap': [True, False]
}

rf = RandomForestClassifier(random_state=42)
# Hyperparameter Tuning
```

```
random_search = RandomizedSearchCV(
    estimator=rf,
    param_distributions=param_dist,
    n_iter=20,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    random_state=42
)

random = random_search.fit(X_train, y_train)
```

```
# Best parameter for training RandomForestClassifier
print("Best Parameters:", random_search.best_params_)
```

```
Best Parameters: {'n_estimators': 400, 'min_samples_split': 15, 'min_samples_leaf': 1, 'max_depth': 20, 'bootstrap': False}
```

```
# Training final model on best parameter
final_model = random_search.best_estimator_
final_model.fit(X_train, y_train)
```

```
RandomForestClassifier
RandomForestClassifier(bootstrap=False, max_depth=20, min_samples_split=15,
                       n_estimators=400, random_state=42)
```

```
from sklearn.model_selection import cross_validate

cv_acc_results = cross_validate(final_model, X_train, y_train, cv=5, scoring='accuracy', return_train_score=True)

print(f"Accuracy Mean: \n Train: {cv_acc_results['train_score'].mean()*100:.3f} \n Validation: {cv_acc_results['test_score
print(f"Accuracy Std: \n Train: {cv_acc_results['train_score'].std()*100:.3f}, \n Validation: {cv_acc_results['test_score'
```

```
Accuracy Mean:
 Train: 98.961
 Validation: 95.790
Accuracy Std:
 Train: 0.500,
 Validation: 6.341
```

Training score before hyperparameter tuning: Training score of 0.99

Training and validation score after hyperparameter Tuninh: Accuracy Mean: Train: 98.96%, this is a sign of overfitting.the model might be memorizing training data.

Validation:On average, the model achieves 95.79% accuracy on unseen validation data.This is quite high, suggesting good generalization—but not perfect.

Accuracy Std: Train: very littel variation in training performance across folds. Validation: 6.341, there is some variation in validation accuracy across different folds.

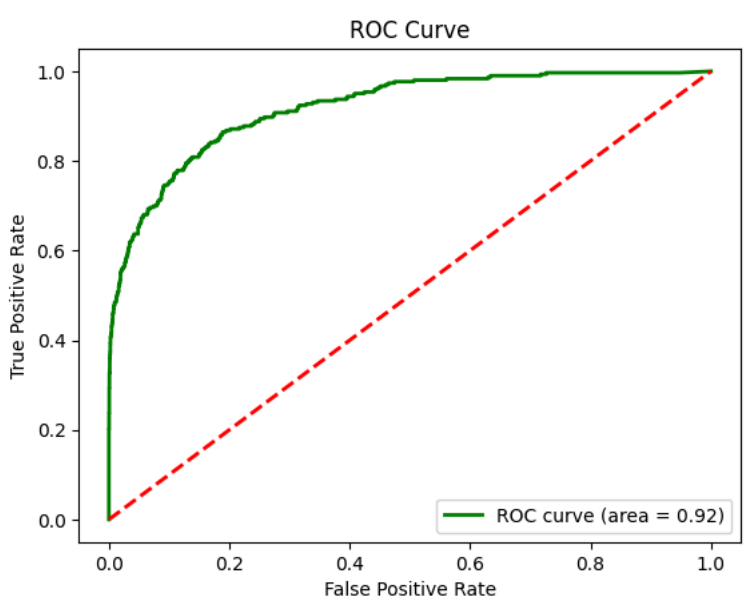
```
# Precision, Recall and F1 Score
from sklearn.metrics import classification_report
y_pred = final_model.predict(X_test)
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.96      0.99      0.97      3488
           1       0.84      0.47      0.60       303

    accuracy                           0.95      3791
   macro avg       0.90      0.73      0.79      3791
weighted avg       0.95      0.95      0.94      3791
```

```
# ROC-AUC Score
from sklearn.metrics import roc_auc_score, roc_curve
y_proba=final_model.predict_proba(X_test)[:,1]
print(roc_auc_score(y_test, y_proba))
```

```
0.9182472863111999
```

```
# ROC-AUC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_proba)
plt.figure()
plt.plot(fpr, tpr, color='green', lw=2, label='ROC curve (area = %0.2f)' % roc_auc_score(y_test, y_proba))
plt.plot([0, 1], [0, 1], color='red', lw=2, linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend(loc="lower right")
plt.show()
```



Random Forest Classifier performance:

The model achieved an overall accuracy of 95%, reflecting strong general performance across the dataset. For the majority class (class 0), the model performs exceptionally well, with a precision of 96%, recall of 99%, and an F1-score of 97%, indicating that it correctly identifies and predicts the majority of negative cases.

However, for the minority class (class 1), the performance is more limited. While the precision is relatively high at 84%, meaning that most predicted positives are correct, the recall is only 47%, showing that the model fails to identify over half of the actual positive cases. The F1-score for class 1 is 0.60, reflecting a weaker balance between precision and recall.

The ROC AUC score is 0.918, which is still very strong — it shows that the model has good overall ability to distinguish between the positive and negative classes.

Ensemble Learning: Gradient Boosting

Gradient Boosting Classifier,it builds a strong predictive model by combining many weak learners (typically shallow decision trees) in a sequential way.

```
from sklearn.ensemble import GradientBoostingClassifier

# Initialize the Gradient Boosting Classifier
gb_clf = GradientBoostingClassifier(random_state=42)

# Train the model
gb_clf.fit(X_train, y_train)
```

```
GradientBoostingClassifier
GradientBoostingClassifier(random_state=42)
```

```
gb_clf.score(X_train, y_train)
```

```
0.9571747586803054
```

Training score is 0.9571 before hyperparamenter tuning.

```
from sklearn.model_selection import RandomizedSearchCV
from sklearn.ensemble import GradientBoostingClassifier

# Define a parameter distribution for RandomizedSearchCV
param_dist_gb = {
    'n_estimators': [100, 200, 300, 400],
    'learning_rate': [0.01, 0.05, 0.1, 0.2],
    'max_depth': [3, 4, 5, 6],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2', None]
}

# Initialize the Gradient Boosting Classifier
gb = GradientBoostingClassifier(random_state=42)

# Initialize RandomizedSearchCV
random_search_gb = RandomizedSearchCV(
    estimator=gb,
    param_distributions=param_dist_gb,
    n_iter=20,
    cv=5,
    scoring='accuracy',
    n_jobs=-1,
    random_state=42
)

# Fit RandomizedSearchCV to the training data
final_gb= random_search_gb.fit(X_train, y_train)

# Print the best parameters and best score
print("Best Parameters:", final_gb.best_params_)
```

```
Best Parameters: {'n_estimators': 200, 'min_samples_split': 2, 'min_samples_leaf': 2, 'max_features': 'sqrt', 'max_depth': 6, 'learning_
```

```
# Get the best parameters from RandomizedSearchCV
best_params = final_gb.best_params_

# Initialize and train the Gradient Boosting Classifier with the best parameters
gb_tuned = GradientBoostingClassifier(random_state=42, **best_params)
gb_tuned.fit(X_train, y_train)
```

```
GradientBoostingClassifier
GradientBoostingClassifier(max_depth=6, max_features='sqrt', min_samples_leaf=2,
                           n_estimators=200, random_state=42)
```

```
# Use the tuned Gradient Boosting Classifier
# Perform cross-validation
cv_results = cross_validate(gb_tuned, X_train, y_train, cv=5, scoring='accuracy', return_train_score=True)

# Print the results
print(f"Cross-Validation Results (Accuracy):")
print(f"Train Score Mean: {cv_results['train_score'].mean()*100:.3f}%")
print(f"Train Score Std:  {cv_results['train_score'].std()*100:.3f}%")
print(f"Validation Score Mean: {cv_results['test_score'].mean()*100:.3f}%")
print(f"Validation Score Std:  {cv_results['test_score'].std()*100:.3f}%")
```

```
Cross-Validation Results (Accuracy):
Train Score Mean: 98.575%
Train Score Std:  0.700%
Validation Score Mean: 95.934%
Validation Score Std:  6.234%
```

Training Score (Before Tuning) Accuracy: 96.75% The model performed well on the training dataset even before tuning.

Cross-Validation Results (After Hyperparameter Tuning): Training Accuracy (98.57%): The model fits the training data very well. Indicates that hyperparameter tuning has improved both training and validation accuracy. Very low standard deviation → consistent performance across training folds.

Validation Accuracy: (95.93%) The model generalizes well to unseen data. A small drop from training accuracy (~3.4%) Std Dev: (6.23%),Indicates performance varies across different folds in cross-validation.

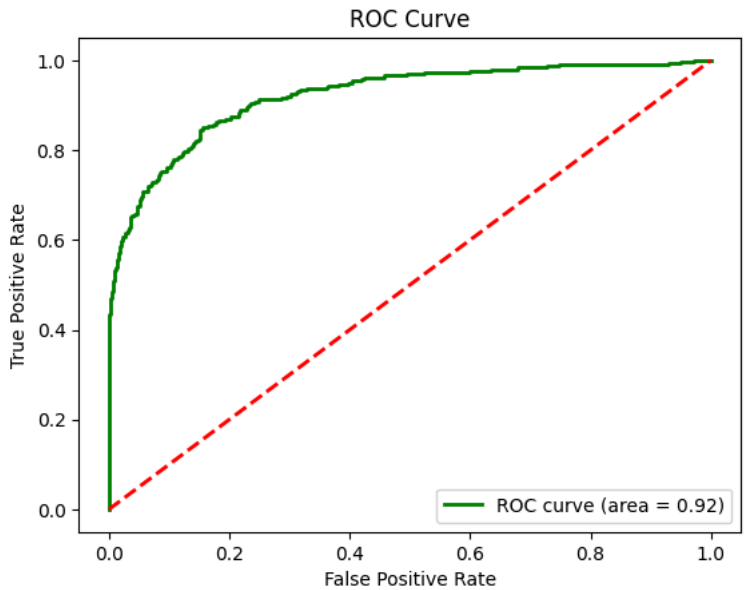
```
from sklearn.metrics import classification_report
y_pred = gb_tuned.predict(X_test)
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.96      0.99      0.97      3488
           1       0.83      0.52      0.64       303

    accuracy                           0.95      3791
   macro avg       0.89      0.75      0.81      3791
weighted avg       0.95      0.95      0.95      3791
```

```
# ROC AUC Score
from sklearn.metrics import roc_auc_score, roc_curve
y_proba=gb_tuned.predict_proba(X_test)[:,1]
print("ROC-AUC Socre: ",roc_auc_score(y_test, y_proba))
```

```
ROC-AUC Socre:  0.9211724498137888
```

```
# ROC AUC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_proba)
plt.figure()
plt.plot(fpr, tpr, color='green', lw=2, label='ROC curve (area = %0.2f)' % roc_auc_score(y_test, y_proba))
plt.plot([0, 1], [0, 1], color='red', lw=2, linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend(loc="lower right")
plt.show()
```



Gradient Boosting Classifier Performance:

The model performs very well on the majority class (class 0), with a precision of 96%, recall of 99%, and an F1-score of 97%.

However, performance on the minority class (class 1) is more limited. While it achieves a precision of 83%, meaning most of its positive predictions are correct, the recall is only 52%, indicating that the model is missing nearly half of the actual positive cases. The F1-score for this class is 0.64, reflecting a moderate balance between precision and recall. the model achieved a ROC AUC score of 0.92, suggesting it has strong discriminative power

Light GBM:

Why I am using LightGBM over XGBoost? LightGBM is faster and more memory-efficient than XGBoost due to its histogram-based and leaf-wise tree growth. It also handles large datasets and categorical features better with similar or higher accuracy.

```
!pip install lightgbm
```

```
Requirement already satisfied: lightgbm in /usr/local/lib/python3.11/dist-packages (4.6.0)
Requirement already satisfied: numpy>=1.17.0 in /usr/local/lib/python3.11/dist-packages (from lightgbm) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages (from lightgbm) (1.16.1)
```

```
import lightgbm as lgb

# Initialize the LightGBM Classifier
lgb_clf = lgb.LGBMClassifier(random_state=42)

# Train the model
lgb_clf.fit(X_train, y_train)
```

```
[LightGBM] [Info] Number of positive: 13882, number of negative: 13882
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.006893 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1846
[LightGBM] [Info] Number of data points in the train set: 27764, number of used features: 35
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
```

```
▾ LGBMClassifier
LGBMClassifier(random_state=42)
```

```
lgb_clf.score(X_train,y_train)
```

```
0.9796499063535513
```

```
import lightgbm as lgb

# Define the parameter distribution for RandomizedSearchCV for LightGBM
```

```
param_dist_lgb = {
    'n_estimators': [100, 200, 300, 400],
    'learning_rate': [0.01, 0.05, 0.1, 0.2],
    'num_leaves': [31, 63, 127],
    'max_depth': [ 5, 6,8,10, 15],
    'min_child_samples': [20, 30, 40, 50],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0],
}

# Initialize RandomizedSearchCV
random_search_lgb = RandomizedSearchCV(
    estimator=lgb_clf,
    param_distributions=param_dist_lgb,
    n_iter=50,  # Number of parameter settings that are sampled. Increase for
    more thorough search
    cv=5,       # Number of cross-validation folds
    scoring='accuracy', # or 'roc_auc', 'f1', etc. depending on your evaluation
    metric
    n_jobs=-1,  # Use all available cores
    random_state=42
)

# Fit RandomizedSearchCV to the training data
random_search_lgb.fit(X_train, y_train)

# Print the best parameters and best score
print("Best Parameters:", random_search_lgb.best_params_)
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
Best Parameters: {'subsample': 0.8, 'num_leaves': 127, 'n_estimators': 300, 'min_child_samples': 20, 'max_depth': 8, 'learning_rate':
```

```
lgb_best_params=random_search_lgb.best_params_
```

```
lgb_tuned = lgb.LGBMClassifier(random_state=42, **lgb_best_params)
lgb_tuned.fit(X_train, y_train)
```

```
[LightGBM] [Info] Number of positive: 13882, number of negative: 13882
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.004022 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1846
[LightGBM] [Info] Number of data points in the train set: 27764, number of used features: 35
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
```

```
LGBMClassifier

LGBMClassifier(max_depth=8, n_estimators=300, num_leaves=127, random_state=42,
               subsample=0.8)
```

```
from sklearn.model_selection import cross_validate
cv_results = cross_validate(lgb_tuned, X_train, y_train, cv=5, scoring='accuracy', return_train_score=True)

print(f"Cross-Validation Results (Accuracy):")
print(f"  Train Score Mean: {cv_results['train_score'].mean()*100:.3f}%")
print(f"  Train Score Std:  {cv_results['train_score'].std()*100:.3f}%")
print(f"  Validation Score Mean: {cv_results['test_score'].mean()*100:.3f}%")
print(f"  Validation Score Std:  {cv_results['test_score'].std()*100:.3f}%")
```

```
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
[LightGBM] [Warning] No further splits with positive gain, best gain: -inf
Cross-Validation Results (Accuracy):
  Train Score Mean: 99.987%
  Train Score Std:  0.008%
  Validation Score Mean: 96.081%
  Validation Score Std:  6.289%
```

LightGBM Model Performance

Before Hyperparameter Tuning Training Accuracy: 97.96% The model already performed very well on the training dataset. Indicates that LightGBM handles the dataset well even with default settings.

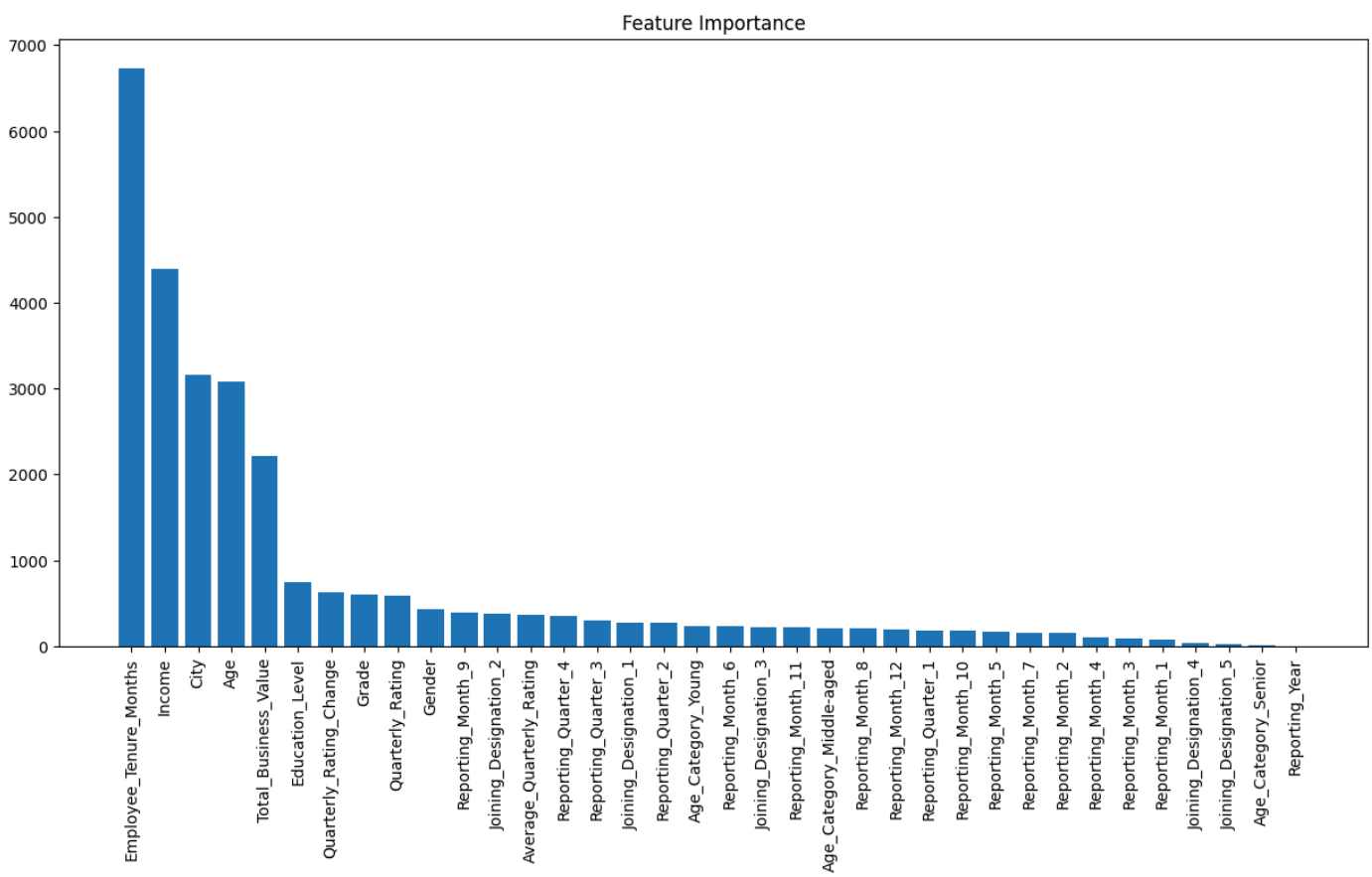
After Hyperparameter Tuning Training Score Mean: 99.98% Improved training accuracy after tuning.

Std Dev: 0.008% → Very low variation across folds, showing consistent model behavior on the training set.

Validation Score Mean: 96.081% Strong generalization to unseen data. Slight drop from training accuracy (~3.1%) is expected and suggests a well-balanced model.

Std Dev: 6.28% → Some variability in validation performance

```
# Important Features of The Model
importances = lgb_tuned.feature_importances_
indices = np.argsort(importances)[::-1]
names = [X.columns[i] for i in indices]

plt.figure(figsize=(15, 7))
plt.title("Feature Importance")
plt.bar(range(X.shape[1]), importances[indices])
plt.xticks(range(X.shape[1]), names, rotation=90)
plt.show()
```



We can obser from grapg that features Employee_Tenure_months, Income,city Age,Total_Business_Value are highly importan feature of the model

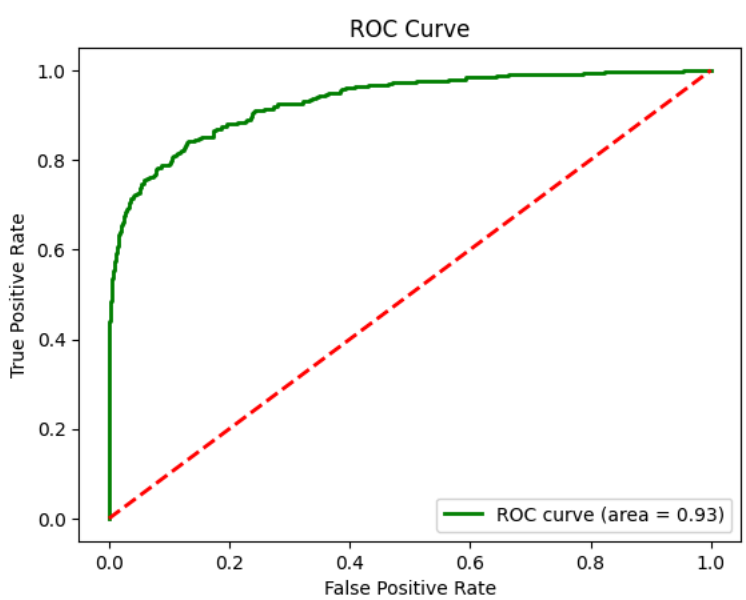
```
# Precision, Recall and f1 score
from sklearn.metrics import classification_report
y_pred = lgb_tuned.predict(X_test)
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.96      0.99      0.98      3488
           1       0.84      0.57      0.68       303

    accuracy                           0.96      3791
   macro avg       0.90      0.78      0.83      3791
weighted avg       0.95      0.96      0.95      3791
```

```
# ROC-AUC Score
from sklearn.metrics import roc_auc_score, roc_curve
y_proba=lgb_tuned.predict_proba(X_test)[:,1]
print("ROC-AUC Socre: ",roc_auc_score(y_test, y_proba))
```

```
ROC-AUC Socre:  0.9312806567353983
```

```
# ROC- AUR Curve
fpr, tpr, thresholds = roc_curve(y_test, y_proba)
plt.figure()
plt.plot(fpr, tpr, color='green', lw=2, label='ROC curve (area = %0.2f)' % roc_auc_score(y_test, y_proba))
plt.plot([0, 1], [0, 1], color='red', lw=2, linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend(loc="lower right")
plt.show()
```



LightGBM Performance:

The LightGBM model achieved a high overall accuracy of 96%, demonstrating strong predictive performance across the dataset. It performs exceptionally well on the majority class (class 0), with a precision of 96%, recall of 99%, and an F1-score of 98% — indicating nearly perfect classification for non-churn (or negative) cases.

For the minority class (class 1), which is typically more challenging to predict, the model achieved a precision of 84%, meaning most predicted positives are correct. More importantly, the recall is 57%, an improvement over previous models — capturing over half of the actual positives. The F1-score for class 1 is 0.68, showing a fair balance between precision and recall.

The model achieved a ROC AUC score of 0.93, indicating excellent ability to distinguish between positive and negative classes across various thresholds.

Observations & Insigts:

Dataset:

1. Older drivers, particularly those in higher grades and designations, are less likely to churn and tend to perform better.
2. Female drivers show slightly better performance and income on average, though gender overall has minimal impact on churn.
3. Higher income is strongly associated with higher grades and retention, with retained drivers contributing significantly more business value.
4. Grade is a critical feature: higher-grade drivers (e.g., Grade 5) earn more, churn less, and deliver more business value.
5. Quarterly rating is a strong predictor of performance and churn: lower ratings are linked with higher churn and lower business value.

6. Shorter tenure (less time with the company) correlates with higher churn, indicating experience impacts retention.
7. Cities like C29, C24, and C22 foster better performance, while C17 and C23 consistently underperform, suggesting regional factors influence driver outcomes.
8. High business value, higher income, and good ratings are interconnected and usually found among retained, high-grade drivers.
9. Education level has little to no impact on churn or performance metrics.

Overall, Grade, Rating, Income, Business Value, and Tenure are the most important drivers of retention and performance.

Model:

Random Forest Classifier: Training Accuracy: 98.96%, Validation: 95.79% → slight overfitting. Class 0: Excellent (Precision 96%, Recall 99%). Class 1: Low recall (47%), F1-score 0.60 → misses many churners. ROC AUC: 0.918 → good class separation.

Insight: Strong overall accuracy, but weak in detecting churners due to class imbalance.

Gradient Boosting Classifier: Training Accuracy: 98.57%, Validation: 95.93% → good generalization. Class 0: Excellent performance. Class 1: Better recall (52%) and F1-score (0.64) than Random Forest. ROC AUC: 0.92 → strong separation.

Insight: Balanced model with better churn detection and stable validation performance.

LightGBM Classifier: Training Accuracy: 99.98%, Validation: 96.08% → high accuracy, slight overfitting. Class 0: Nearly perfect. Class 1: Best recall (57%) and F1-score (0.68) among all models. ROC AUC: 0.93 → highest discrimination power.

Insight: Best overall performer; combines accuracy with strong churn prediction.

Light GBM performed Best, Best at identifying churners (highest recall and F1-score for class 1). Highest ROC AUC → Strongest ability to distinguish between churn and non-churn. Consistently high validation accuracy with low variation.

```
Start coding or generate with AI.
```

Questionnaire:

1. **What percentage of drivers have received a quarterly rating of 5?**

   The dataset does not contain any drivers with a Quarterly Rating of 5. The highest rating observed is 4, which is achieved by only around 10% of the drivers.

2. **Correlation between Age and Quarterly Rating.**

   The correlation between Age and Quarterly Rating is approximately 0.17, indicating a weak positive relationship. This means that older drivers tend to receive slightly higher quarterly ratings, but the association is not strong.

3. **Name the city which showed the most improvement in Quarterly Rating over the past year.**

   The city that showed the most improvement in Quarterly Rating over the past year is City C29, with an increase of 0.201 points (from 2.0688 in 2019 to 2.2699 in 2020).

4. **Drivers with a Grade of 'A' are more likely to have a higher Total Business Value.**

   The dataset does not have a Grade 'A', but assuming Grade 5 represents 'A', drivers with a Grade of 5 are more likely to have a higher Total Business Value. They are top-tier performers, generating the highest average revenue (₹1.41M), with some achieving exceptionally high values.

5. **If a driver's Quarterly Rating drops significantly, how does it impact their Total Business Value in the subsequent period?**

   If a driver's Quarterly Rating drops significantly, it generally leads to a decrease in their Total Business Value in the subsequent period. Drivers with rating drops experienced an average loss of ₹1.48 lakhs in Total Business Value. The median loss was around ₹0.93 lakhs, showing a consistent downward trend. 75% of these drivers saw no significant gain, and 25% lost more than ₹2.99 lakhs. While a few outliers maintained or increased their business value, the overall pattern suggests a negative impact.

6. **FromOla's perspective, which metric should be the primary focus for driver retention?**

   Answer is Recall. Recall measures the ability to identify actual churners- drivers who are truly at risk of leaving. If Ola misses these drivers (called false negatives), they lose the chance to take action and stop them from leaving. Even if some false positives occur (predicting churn where there isn't), it's better to act early than miss a potential loss.

7. **How does the gap in precision and recall affect Ola's relationship with its drivers and customers?**

   Low Recall (missed churners): Ola fails to identify drivers who are planning to leave. This means they miss the chance to support or retain them, leading to driver shortages, longer wait times, and unhappy customers.

Low Precision (false alarms): Ola wrongly predicts some drivers as churn risks. These drivers may be contacted unnecessarily, which can annoy loyal drivers or waste company resources.

8. **Besides the obvious features like "Number of Rides", which lesser-discussed features might have a strong impact on a driver's Quarterly Rating?**

   Ride Cancellation Rate: High cancellations may signal unreliability or poor service, negatively impacting ratings

   On-Time Arrival: Regular delays can frustrate customers, reducing ratings.

   Complaint Frequency: Number or frequency of passenger complaints—even if resolved—can weigh on perceived service.

   Driver's Age & Experience: Older or more experienced drivers may handle situations better, potentially earning higher ratings.

   Vehicle Condition Score / Age of Vehicle: Cleaner, newer, or well-maintained vehicles tend to improve the passenger experience.

9. **Will the driver's performance be affected by the City they operate in? (Yes/No)**

   City has a significant impact on driver performance. Cities like C29, C24, and C22 foster high-performing drivers, reflected through better Rating, higher business value, and stronger grades. In contrast, cities like C17 and C23 consistently underperform across key metrics—possibly due to limitations in infrastructure, demand, or operational management.

   Performance is deeply interconnected: Higher business and income often lead to better driver motivation, well-being, and vehicle maintenance. These in turn influence customer feedback, driver grade, and even job designation. Such positive cycles are more common in large metro cities, where opportunities are abundant. Meanwhile, small towns may struggle with limited demand and support systems, hindering overall performance outcomes.

10. **Analyze any seasonality in the driver's ratings. Do certain times of the year correspond to higher or lower ratings, and why might that be?**

    Driver Rating Seasonality:

    March (2019 & 2020): Ratings peak in this month (2.10 in Mar 2019, 2.09 in Mar 2020), possibly due to festive seasons or performance incentives.

    Start and End of Year: Ratings dip around January and December (e.g., 1.95 in Jan/Dec 2019 and ~1.99 in Dec 2020), potentially due to holidays and reduced activity.

    Mid-2020 (May–June): A noticeable drop in ratings, likely due to the COVID-19 impact. This period also saw a sharp decline in the number of rides.

    July to October 2020: Ratings remained relatively stable and slightly higher, possibly reflecting recovery after COVID as ride demand increased.

Recommendations:

1. Prioritize High-Risk Churners Using Recall-Focused Models Since missing a churner has a greater cost than acting on a false prediction, Ola should prioritize models with high recall, like LightGM.
2. Target Drivers with Low Quarterly Ratings for Early Intervention Significant drops in quarterly ratings lead to noticeable losses in business value.
3. Focus on High-Value Drivers: Grade 5, High Income, Longer Tenure Drivers with higher grades and longer experience consistently show better performance, retention, and revenue.
4. Enhance Support in Underperforming Cities Cities like C17 and C23 show poor outcomes. Investigate regional issues (infrastructure, demand, support) and consider deploying location-specific strategies like flexible pricing, training hubs, or city-level incentives.
5. Use Performance-Linked Metrics Beyond Rides Include cancellation rate, complaint frequency, on-time arrival, and vehicle condition in evaluating and coaching drivers.
6. Support Newer Drivers More Aggressively Newer drivers churn more often. Implement onboarding support, mentorship programs, or frequent check-ins in the first 3–6 months to reduce early exits.
7. Gender-Neutral Strategy, Age-Focused Support Gender does not majorly affect churn, but older drivers perform better.
8. Reward Consistency in Business Value and Income Retained drivers contribute significantly more to business value. Design rewards tied to consistent income contribution, encouraging retention through financial motivation.
9. Data-Driven Policy Adjustments Continuously monitor Grade, Income, Tenure, Rating, and Business Value as key drivers of churn.

Start coding or generate with AI.